

Stage

Technical build specification, version 2

Stack, demo, compute and storage answers, plus the v2 upgrade: sound, separation, scope and Spotify

Companion to the Stage project plan

Prepared by Daniel (Tesh) | November 2026 | Facts verified November 2026

Stage is a working name. This is an independent proposal and is not affiliated with or endorsed by Spotify.
Prices, licenses and quotas change; recheck anything that matters before relying on it.

Contents

1 About this document 4

2 The stack at a glance 6

3 Spotify's world and how Stage fits 8

4 Two scales: demo and platform 10

5 Development environment and project structure 12

6 Data formats and contracts 15

7 Subsystem 1: stem separation 18

8 Subsystem 2: analysis and mix intelligence 23

9 Subsystem 3: the spatial renderer 26

10 Back end: orchestration, services and reliability 29

11 Front end and user experience 32

12 The compute question 36

13 The storage question 39

14 Testing and quality assurance 42

15 The demo and how to show it 45

16 Licensing, rights and responsible use 48

17 Risks and open questions 50

18 The build sequence 52

19 Version 2: what changed and why 54

20 Scope made simple: one page, three tiers, five slices 56

21 The sound: Pan-Anchored Spatial Rendering 60

22 Separation v2: better results with far less compute 67

23 Languages and stack for a light, portable engine 74

24 The demo v2: built to impress in five minutes 77

25 Reaching Spotify: a realistic strategy 79

26 Experiments, gates and the v2 roadmap 81

A Appendix A: glossary 84

B Appendix B: one-page checklists 85

C Appendix C: sources 86

D Appendix D: sources added in v2 87

1 About this document

The first document explained *what* Stage is and *why* it matters. This one explains *how to build it*: which tools to use, how the pieces connect, how it would fit into Spotify's systems, and how to answer the hard questions about compute and storage before anyone has to ask them.

It is written like an engineering specification. Every important choice has the same four parts: the choice, why it was made, what it costs, and what else was considered. That way you can defend any decision in a conversation, or change it later without breaking the rest of the plan.

How this was checked

You asked for rigorous research, so the facts in this document were verified against current sources in November 2026 where possible. Three kinds of statements appear, and the document says which is which.

Kind	How it is marked	Example
Verified	Stated plainly with a source in Appendix C.	Spotify's Q3 2026 subscriber count, Google Cloud Storage list prices.
Estimate	Marked as an estimate, with the calculation shown.	Size of a 3.5 minute FLAC render.
Open question	Marked as unverified, with a way to settle it.	Whether the pretrained Demucs weights may be used commercially.

> **Research changed the plan**
>
> Several findings changed the plan while researching it. The most important: the original Demucs repository was archived and is no longer maintained; the license status of some pretrained separation weights is unclear for commercial use; and Android's built-in spatializer exists but depends on each phone maker supplying the effect engine. All three are handled in the chapters that follow.

> **Version 2 additions**
>
> This is **version 2**. Chapters 1 to 18 are the original specification. Chapters 19 to 26 add new methods for the sound, the separation, the scope and the approach to Spotify, found through further research. Where they differ, Chapters 19 to 26 take precedence; nothing in Chapters 1 to 18 has been removed.

Who this document is for

- **You**, as the builder: it is a work plan you can start from on day one.
- **A technical reader at Spotify or elsewhere**: it shows that the proposal has been thought through at the level of data formats, failure handling and cost.
- **A mentor or collaborator**: it makes it easy to review one decision at a time.

How the document is organised

Chapters	Topic
2 to 4	The decisions at a glance, how Stage fits Spotify's world, and the two scales it has to work at.
5 and 6	The working environment, the code layout and the data formats everything shares.
7 to 9	The three engine subsystems: separation, mix intelligence, rendering.
10 and 11	Back end and front end, including the interface and the demo player.
12 and 13	The two cost questions: compute and storage, with numbers and a ready answer for each.
14 to 16	Testing, the demo and how to show it, and the licensing and security picture.
17 and 18	Risks and open questions, then the order in which to build.

2 The stack at a glance

This is the whole plan on two pages. Each row is explained and justified in its own chapter. The columns separate what is used to build the **demo** (what you can build with a laptop and free services) from what the same layer would become in a **platform** deployment at Spotify's scale.

Layer	Demo choice	Platform choice	Main reason
Language (engine)	Python 3.12	Python for models, Rust or C++ for hot loops	Audio and ML ecosystem is in Python. Rust or C++ only where speed is proven necessary.
Deep learning	PyTorch (CPU on laptop, free GPU for training)	PyTorch, exported to ONNX for serving	Separation models are PyTorch-native. ONNX gives portable, faster inference.
Separation	HT-Demucs family plus a RoFormer-class model for comparison	Ensemble tuned per style, fine-tuned on licensed data	Best open baselines today. Gains come from ensembling and fine-tuning.
Audio I/O	soundfile and FFmpeg, 24-bit FLAC	Same, inside existing ingest	Matches Spotify's delivery format exactly.
Analysis	librosa, a small classifier	Same models served in the batch graph	Mature, free, well documented.
Rendering	NumPy and SciPy prototype, then a Rust or C++ core	Rust or C++ library in the batch job	Correctness first, then speed.
HRTF data	Open SOFA datasets	Licensed or measured set plus personalisation	Free and standard now; better data later.
Job system	Local queue	Beam-style batch pipeline (Klio is Spotify's own audio framework)	Same graph, different scale.
Storage	Folders and JSON	Object storage with tiering	Mirrors a real layout; cheap at scale.
Back-end API	None needed (static files)	Small service in front of the registry and catalog	Keeps the demo free and simple.
Web front end	TypeScript, Vite, Svelte, Web Audio API	Not applicable	Lightweight, shareable by link, no install.
Mobile client	Optional Android build with Media3	Spotify's own apps	Shows the real-device path.
Testing	pytest, golden audio files, objective metrics, blind listening	Same, plus pipeline-level monitoring	Audio needs measurement, not just unit tests.
CI and hosting	Git, GitHub Actions, free static hosting	Spotify's own	Free and enough for a public demo.

Design philosophy behind these choices

1. **Prove it with measurements.** Any claim in the pitch, such as cleaner separation or fidelity to the original, has a metric behind it.

2. **Keep the interfaces boring and stable.** Subsystems talk through files and JSON. That makes each piece testable alone and replaceable later.
3. **Use the slow, clear version first.** Optimise only after a correct version exists and a measurement shows where time goes.
4. **Choose open and free, but check the license.** Free to download does not always mean free to use commercially.
5. **Plan for the failure, not only the success.** Every stage can fail, and the system must fall back to ordinary stereo without any listener noticing.

3 Spotify's world and how Stage fits

To build something that could plug in, you need an accurate picture of the host system. What is public is summarised below. Spotify's internal details are not public, so anything beyond this is a design assumption and is marked as one.

What is publicly known

Topic	What is known	Why it matters for Stage
Delivery format	One stereo master per track, FLAC preferred, WAV accepted. Internally reduced to at most 44.1 kHz and 24-bit FLAC.	Stage's input is already defined. Its output can use the same format.
Listener quality	Lossless 24-bit/44.1 kHz FLAC for Premium since September 2025. Bluetooth cannot carry full lossless.	A spatial render can be lossless too; Bluetooth limits still apply.
Scale	306 million Premium subscribers and 788 million monthly users in Q3 2026.	Anything per-track must be cheap and automatic.
Audio-processing tooling	Spotify built and open-sourced Klio, a Python framework on Apache Beam for audio pipelines, so research and production share tooling.	The batch design in this plan matches how Spotify already processes audio at scale.
Audio effects library	Spotify's Audio Intelligence Lab built and open-sourced Pedalboard, which includes convolution and reverb.	A natural tool for the renderer; note it is released under the GPL v3 license.
Cloud	Google Cloud is Spotify's long-time platform.	Cost figures in this document use Google Cloud list prices.

> **Check before relying on Klio**
>
> Klio was announced in 2020 as a framework built for audio and binary file processing, with an emphasis on processing new content as it arrives. Whether it is still the preferred internal tool today is not public, so the plan treats it as proof that the architecture is natural for Spotify, not as a dependency.

The integration surface

A well-designed add-on touches few things. Stage touches the following, and nothing else.

Surface	Interaction	Interface contract
Ingest event	Stage subscribes to the event that fires when a master is accepted.	Track id, audio fingerprint, location of the master.
Consent registry	Stage asks whether the track's rights holder opted in and with which options.	Track id in; allowed modes, intensity cap and approval rule out.
Catalog metadata	Stage writes a small block describing the spatial assets.	Status, list of modes, asset ids, loudness of each render.
Asset storage	Stage writes stems, renders and reports to its own area.	Stable paths keyed by track id and version.
Encoding ladder	A render is passed to the existing encoder like any other stereo master.	24-bit PCM or FLAC stereo, 44.1 kHz.

Surface	Interaction	Interface contract
Client	The app reads the metadata block and requests the chosen asset.	One new field on the track object and one toggle in the UI.

Two design points that matter

Reusing the encoding ladder

Because a spatial render is ordinary stereo audio, Spotify's existing pipeline can encode it into every lower-bitrate version it already produces. This means listeners on limited data or weak connections could also get spatial audio, at a lower bitrate, without any new codec. Whether heavy lossy compression damages the spatial cues is an open question to measure in the demo (Chapter 14), but the design makes the test easy.

Not double-processing on Android

Android 13 and later include a standard spatializer in the audio framework, but the phone maker has to supply the effect engine for it to work, and the framework lets an app mark a stream as already spatialised so the system does not process it again. Stage's client must set that flag on pre-rendered streams, otherwise a phone with its own spatializer could apply a second effect on top and ruin the mix. This is a concrete example of why the engine must be owned by the app: the platform feature exists, but it is not uniformly available.

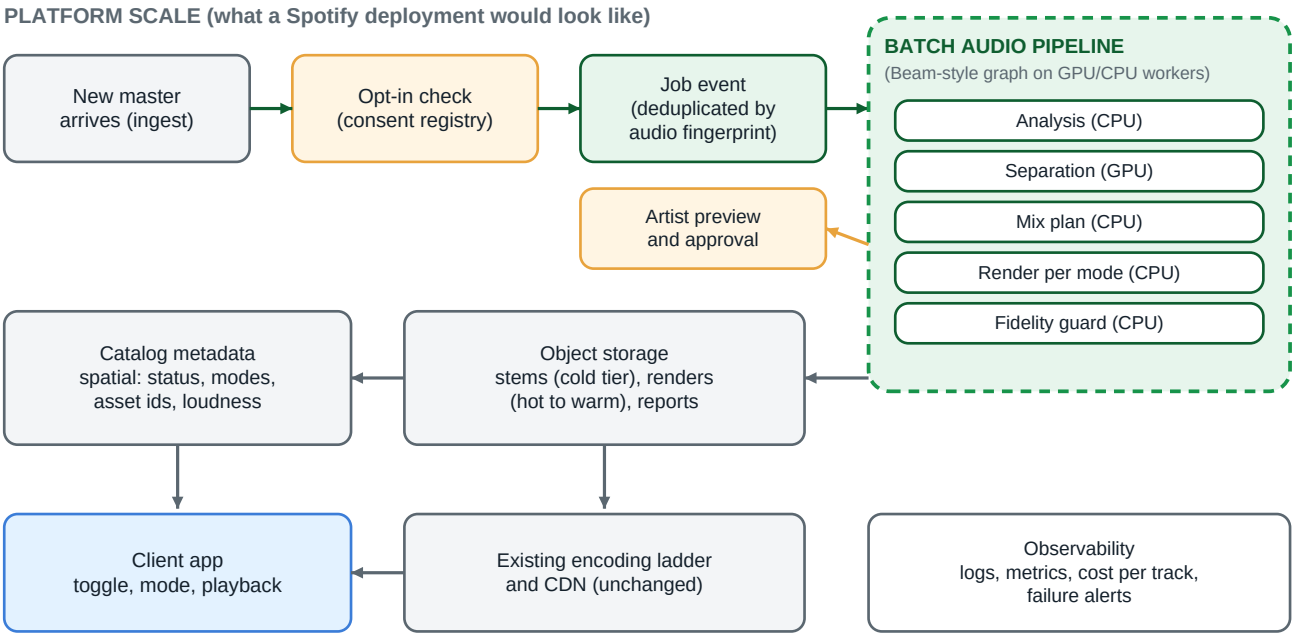


Figure 1. Stage inside a Spotify-scale deployment. Existing systems are grey; the new pipeline is green; amber steps involve the rights holder.

4 Two scales: demo and platform

Stage has to be believable at two very different scales. The architecture is deliberately the same at both, so that what you build on a laptop is a faithful small version of what a company would run.

Requirements

Functional requirements

1. Take one stereo master and produce, for each requested mode, a spatial stereo file of the same duration and sample rate.
2. Keep each render time-aligned with the original so listeners can switch without a jump.
3. Produce a plan file and a quality report for every render.
4. Refuse to publish a render that fails the fidelity checks, and fall back to the normal stereo track.
5. Respect the rights holder's options: opt-in, intensity cap, protected elements, approval.
6. Let a listener switch between stereo and spatial, and between modes, in the player.

Non-functional requirements

Quality attribute	Target	How it is tested
Fidelity	Downmix, low-end and loudness checks pass on the whole test library.	Automated guard on every render.
Alignment	Spatial and original aligned to within about 1 millisecond.	Cross-correlation test.
Reproducibility	Same input and settings give bit-identical output on the same machine.	Golden-file tests.
Robustness	Any failure leaves the track in stereo and logs the reason.	Fault-injection tests.
Throughput (platform)	Release-day spikes absorbed through a queue and extra workers.	Load simulation.
Cost	Compute and storage cost per track is measured and published.	Per-job metrics.
Data use (demo)	Demo page loads fast on a poor connection; heavy files only on request.	Network throttling tests.
Accessibility	All controls keyboard and screen-reader usable.	Manual and automated audits.

Same design, two sizes

Aspect	Demo	Platform
Work unit	One track at a time from a command line	One job per opted-in track from an event
Parallelism	One process, or a few on the laptop	Many workers, GPU for separation
State	Folders and JSON files	Object storage plus a metadata service
Consent	A local config file that simulates the registry	The real registry
Failure handling	Log and skip	Retry with backoff, then flag for review
Monitoring	Log files and a summary report	Metrics, dashboards, alerts
Cost	Free tools and your own electricity	Measured per track and budgeted

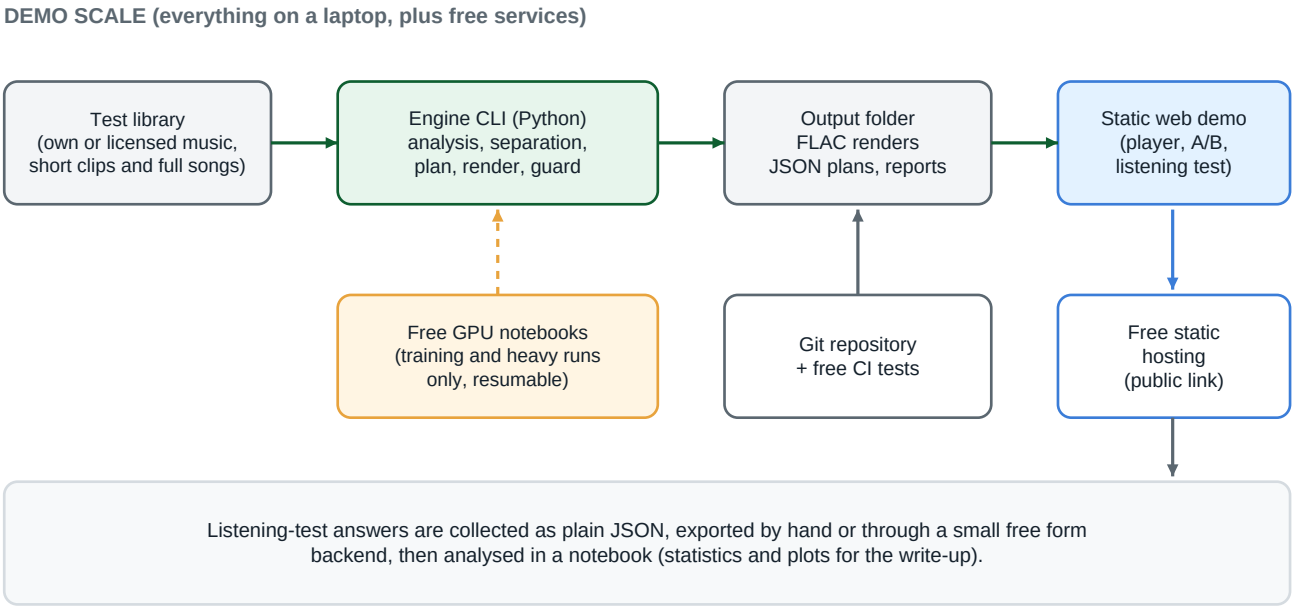


Figure 2. The demo architecture. It reproduces each platform stage at small scale, using free tools.

What the demo deliberately leaves out

- Real-time rendering on the phone, head tracking, and speaker or car rendering. They are real features, but they are a later generation of the product.
- Integration with Spotify's real systems. The demo imitates the interfaces and documents them, which is the honest way to show the fit.
- A user account system. The demo is a public page with no login, which also removes privacy risk.

5 Development environment and project structure

A project this size succeeds or fails on its working habits. This chapter fixes the tools, the folder layout and the rules for working, so that the build stays organised from the first day.

Tools

Need	Tool	Why	Alternative
Editor	VS Code	Free, strong Python, C++ and TypeScript support, one editor for every part.	PyCharm for Python, CLion for C++.
Python environments	uv or venv with a locked requirements file	Reproducible installs; uv is fast and light.	conda (heavier download), poetry.
Version control	Git with GitHub	Free, standard, hosts CI and the public project page.	GitLab, Codeberg.
Large files	Git LFS or DVC for models and test audio, or keep audio outside the repo	Audio must not bloat the repository.	Store only scripts that download and verify the data.
Notebooks	Jupyter in VS Code for experiments; scripts for anything repeated	Notebooks for exploring, scripts for results you must reproduce.	Plain scripts only.
Task runner	A single Makefile or a small Python task file	One command per job, easy to document.	Just, nox.
Formatting and linting	ruff and black for Python, clang-format for C++, ESLint and Prettier for web	Consistent style without arguments.	flake8, pylint.
Testing	pytest for Python, a small test harness for C++, Vitest for web	Fast, simple, good plugins.	unittest, Catch2 for C++.
Audio tools	FFmpeg, SoX, a loudness meter, a spectrum viewer, Audacity or Reaper trial for listening	Needed to inspect and compare results by ear and by eye.	Any DAW you already own.

A realistic data budget

Mobile data is a real constraint, so the plan treats downloads as a budget to be spent once, on a good connection, and never repeated.

Download	Approximate size	When
Python packages (PyTorch CPU build, audio libraries)	A few hundred megabytes in total (check the exact size before downloading)	Once, on Wi-Fi.
Separation model weights	Tens to a few hundred megabytes each; one model of the HT-Demucs family is on the order of 84 MB	Once per model, then cached.

Download	Approximate size	When
HRTF datasets (SOFA files)	Small per subject; start with one or two subjects	Once.
Test music	Your own files and licensed or openly licensed material	Local only.
Web tooling (Node packages)	Tens to hundreds of megabytes	Once; keep the lockfile.

- Keep a local cache folder for models and datasets, and back it up so a reinstall costs no data.
- Prefer packages that ship small inference-only builds. For example, an inference-only Demucs package exists and avoids the training dependencies.
- Use free GPU notebooks for heavy runs, so large intermediate files never touch your connection or disk.

Repository layout

One repository with clear boundaries. Each folder can be understood and tested on its own.

```
stage/
  engine/ Python package: the audio engine
    io/ reading, writing, resampling, loudness
    analysis/ features and style classification
    separation/ model wrappers, ensembling, residual
    mixplan/ placement policy, templates, plan writer
    render/ HRTF loading, convolution, room, bass, limiter
    guard/ fidelity checks and quality report
    cli.py command line entry points
  core-dsp/ Rust or C++ library for hot loops (later)
  web/ TypeScript front end (player and listening test)
  android/ optional Kotlin client
  configs/ templates, modes, thresholds (YAML)
  schemas/ JSON schemas for plan, report, consent
  tests/ unit tests, golden files, property tests
  data/ (not committed) clips, stems, outputs
  docs/ these documents and decision records
  scripts/ download, verify, benchmark
```

Working rules

1. **Every decision gets a short written record** in the docs folder: the choice, why, drawbacks, alternatives. This document is the first batch.
2. **Every result is reproducible from a command.** If a number appears in the write-up, a script regenerates it.
3. **Small commits with clear messages.** It also becomes your public development history.
4. **No audio in the repository.** Keep a manifest of files with checksums so anyone can reproduce your test library from licensed sources.
5. **Branch per experiment.** Merge only what is measured to be better.
6. **Comments explain reasoning, not syntax.** Code comments say why a choice was made, because that is what is hard to recover later.

Decision: one repository, several packages	
Choice	A single repository (a monorepo) holding the engine, the web front end and the optional Android client.
Why	Shared schemas and configs stay in sync across all parts. One place to read, one history, one issue list. It is much easier for a visitor to understand.
Drawbacks	It can grow large, and separate parts cannot be released on separate schedules.
Alternatives	Separate repositories per part (cleaner boundaries, more overhead). A monorepo is the better fit for a project with one author and tightly linked parts.

6 Data formats and contracts

Subsystems cooperate only through well-defined files. This is the most important discipline in the whole design. If the formats are clear, you can swap the separator, rewrite the renderer in another language or change the front end without breaking anything else.

Audio conventions

Item	Rule	Reason
Container and codec	FLAC for stored audio. WAV is allowed inside the pipeline as a scratch format.	Matches Spotify's own delivery and streaming format and halves storage compared with raw PCM.
Sample rate	44.1 kHz throughout. Resample only at the input if a file differs.	Matches the platform maximum and avoids repeated conversions.
Bit depth	Compute in 32-bit float. Store 24-bit.	Float avoids rounding problems during processing; 24-bit matches the delivery.
Channels	Stereo in and stereo out. Mono sources are duplicated to two channels at input.	A uniform interface for the rest of the system.
Duration	Output length equals input length, sample for sample.	Needed for seamless switching and easy validation.
Alignment	Group delay of filters is compensated so that content stays aligned.	A render must not shift in time against the original.
Loudness	Integrated loudness measured by ITU-R BS.1770 and recorded for every render.	The platform normalises loudness, so the render must match its original.
Dither	Apply once, at the final conversion to 24-bit, if at all.	Avoids stacking noise.

Identifiers

- **Track id**: the platform's own identifier, passed through unchanged.
- **Audio fingerprint**: a hash of the decoded audio. Used to detect when two track records share the same master, so the work is done once.
- **Run id**: a unique id for each processing run, linked to the versions of the model, the configs and the code that produced it.
- **Version tuple**: engine version, separator version, policy version, HRTF set version. Stored with every output so any result can be reproduced or invalidated.

The plan file

The mix plan is a human-readable JSON file. It is the contract between the mix-intelligence subsystem and the renderer, and it is what an artist would be shown to understand what was done. A simplified example follows. The real schema lives in the schemas folder and is checked automatically.

```
{ "track_id": "demo-001", "engine": "0.3.0", "style": {"afrobeats": 0.71, "pop":
    0.18, "other": 0.11},
  "mode": "studio_room", "intensity": 0.6,
  "stems": [
    {"name": "vocals_lead", "az": 0, "el": 0, "dist": 1.2, "spread": 0.1,
        "room_send": 0.15, "orig_blend": 0.35},
    {"name": "drums", "az": 0, "el": 0, "dist": 1.5, "spread": 0.3,
        "room_send": 0.20, "orig_blend": 0.40},
    {"name": "perc", "az": 70, "el": 0, "dist": 2.0, "spread": 0.5,
        "room_send": 0.30, "orig_blend": 0.20},
    {"name": "bass", "az": 0, "el": 0, "dist": 1.0, "spread": 0.0,
        "room_send": 0.00, "orig_blend": 0.60, "mono_below_hz": 120}
  ],
  "room": {"rt60_s": 0.45, "predelay_ms": 12}, "protected": ["vocals_lead:centre"] }
```

The numbers in this example are illustrative. The important points are the structure and the fact that every placement parameter, including how much of the original stays in the mix, is explicit and reviewable.

The quality report

Field	Meaning
alignment_ms	Lag between original and render from cross-correlation. Must be within tolerance.
downmix_db	Difference after folding the render back to stereo, by frequency band.
low_end_corr	Similarity of the region below roughly 120 Hz.
loudness_lufs	Integrated loudness of original and render, and their difference.
true_peak_db	Highest inter-sample peak.
mono_cancel_db	Level lost when summing to mono.
artifact_flags	Time ranges where a glitch or sudden change was detected.
verdict	pass, fail or review, with reasons.
versions	The version tuple used to produce the result.

The consent record

Field	Meaning
track_id	The track this record applies to.
opted_in	True or false. Default false.
agreement_ref	Reference to the signed agreement that authorises processing.
modes_allowed	Which modes may be offered.
intensity_cap	Maximum intensity from 0 to 1.
protected	Elements that must not be moved.
approval	none, required, or approved with timestamp and approver.
withdrawn_at	If set, the spatial assets are removed and the track reverts to stereo.

Repository of examples

Keep a small folder of example files, one valid and one invalid for each schema. They serve as documentation, as test data and as a starting point for anyone who wants to build against your formats.

Decision: JSON files as the interface between stages	
Choice	Each stage reads and writes JSON and FLAC files with schemas that are validated on read and write.
Why	Human-readable, easy to diff, easy to test, language independent, and a good fit for the artist-approval story. It works the same on a laptop and in a cloud pipeline.
Drawbacks	Slightly more verbose than a binary format. Large numeric arrays do not belong in JSON.
Alternatives	Protocol Buffers (compact, typed, harder to read by eye) or a database (more moving parts). Protocol Buffers could replace JSON inside a platform deployment without changing the logical design.

7 Subsystem 1: stem separation

Separation decides how clean the parts are, and therefore how convincing the final mix can be. This chapter chooses the models, designs a pipeline that improves on a single model, and plans the training and measurement work, with the licensing facts that came out of the research.

What was verified about the current model landscape

- **Demucs is no longer actively maintained upstream.** The original repository from Meta is archived and read-only; its author, who left Meta, points users to a personal fork, and packaging forks that keep inference working with newer PyTorch also exist.
- **The strongest published open models are now RoFormer-class.** A Mel-Band RoFormer paper reports that a Band-Split RoFormer trained on MUSDB18-HQ plus 500 extra songs reached an average SDR of 11.99 dB, and community model cards claim clear gains over HT-Demucs on vocals.
- **Demucs is still very good and very practical.** On MVSEP's multisong benchmark, the fine-tuned HT-Demucs scores about 12.05 dB on bass, 11.24 on drums and 8.33 on vocals, but only 5.74 on the catch-all 'other' stem.
- **The 'other' stem is the weak point.** The six-source Demucs variant adds guitar and piano but scores only 2.63 dB on 'other' in the same benchmark, and the project's own notes say its piano output does not work well.
- **Weights have mixed licensing.** Demucs code is MIT-licensed. For the pretrained weights, the original project notes said they were for research purposes, citing the dataset terms, while some Hugging Face copies carry an MIT tag; a user publicly asked whether that tag is intentional and the question was open at the time of writing. Several RoFormer weights list their license as unknown.

> **The license question**
>
> Do not assume that a downloadable pretrained model may be used in a product. For the demo and for research it is normal practice. For a commercial deployment, the weights need a clear license or must be replaced by weights trained on data you are allowed to use. This is why the plan includes a fine-tuning stage with licensed data: it is both a quality step and a legal one.

Model options compared

Option	Strengths	Weaknesses	Role in Stage
HT-Demucs (4 stems)	Fast, well documented, stable tooling, ONNX exports exist, permissive code license.	Weights license unclear for commercial use; 'other' stem weak.	Baseline and fallback.
HT-Demucs fine-tuned (bag of four specialists)	Best Demucs quality, one specialist per stem.	Roughly four times the compute of the base model.	Quality tier for showcase tracks.
HT-Demucs 6-stem	Offers guitar and piano.	Weak on piano and 'other'.	Experiment only.
Band-Split or Mel-Band RoFormer	Top scores on vocals and strong overall, an active community and training framework.	Larger and slower; weight licenses often unclear.	Second opinion and ensemble member.
Open-Unmix and similar	Small, simple, clear license history.	Lower quality than current leaders.	Teaching baseline and licensing-safe fallback.
Your own fine-tuned model	License you control; tuned to your genres.	Needs data and GPU time; may overfit on small data.	Long-term goal.

Pipeline design: better than any single model

The plan does not simply run one model. It treats separation as a small system with four steps, each of which can be measured.

1. **Primary separation.** Run the chosen model with overlapping segments. Hybrid Transformer Demucs works on 7.8-second segments, so long songs are cut into overlapping chunks and blended back together with a smooth window to avoid seams.
2. **Second opinion.** Run a different model family on the same audio and combine the two estimates per stem with weights chosen from validation results. Different architectures make different mistakes, so averaging reduces artifacts.
3. **Consistency step.** Make sure the stems and a residual add back to the original exactly (below). Nothing is lost.
4. **Targeted cleanup.** Apply special handling where the baseline is weak: split the 'other' stem further, and reduce bleed in the lead vocal.

The consistency step

Let x be the input mix, and s1 to sN the separated stems. Define the residual r as what the stems fail to explain. There are two ways to keep the total honest, and the plan tests both by ear.

```
Option A (keep the residual as its own stem):
    r = x - (s1 + s2 + ... + sN)
    stems = s1 ... sN plus r (they sum to x exactly)

Option B (share the residual in proportion to each stem's local energy):
    s_i' = s_i + r * |s_i|^2 / (|s_1|^2 + ... + |s_N|^2) per time-frequency bin
    (the s_i' sum to x exactly, and no extra stem is needed)
```

Option A is transparent and lets the renderer decide how much of the leftover to place. Option B is simpler to render but can push leftover noise into whichever stem is loudest. Start with Option A.

Cleaning the weak stems

- **Split 'other' into texture and rhythm.** Harmonic-percussive separation (available in standard audio libraries) pulls sustained sounds such as pads away from transient sounds such as plucks and percussion hits. It is cheap and requires no training.
- **Use stereo information.** A mid and side analysis of 'other' separates what is centred from what is wide, which is exactly the information placement needs.
- **Specialist models for drum parts.** Community models exist that split drums into kick, snare and cymbals. Their quality and licenses need checking before use.
- **Vocal bleed reduction.** A light post-filter driven by the drum and bass stems can reduce hi-hat and kick leakage in the vocal stem. It must be tested for audible side effects, such as lost consonants.

Fine-tuning plan

The goal is a model that behaves better on the music you care about and whose license you control. Fine-tuning means continuing training of an existing model on new data, which needs far less compute than training from scratch.

Step	Detail	Reason
Data	Multitrack sessions you own or have permission to use, plus openly licensed datasets. Aim for balance across genres, including African popular music.	Quality and legality both come from the data.
Augmentation	Remix stems from different songs, vary gain, apply pitch and tempo shifts, add room and compression variants.	Creates many training examples from few songs.
Loss	A time-domain error plus a multi-resolution spectral error, evaluated on the stems and on the sum.	Matches how models in this family are trained and rewards both waveform and spectral accuracy.
What to train	Start by training only the final layers, then unfreeze more if validation improves.	Lower risk of overfitting and lower compute.
Where to train	Free GPU notebooks in resumable sessions of a few hours, with a checkpoint saved to storage after every short interval.	Free tiers have time limits and may disconnect.
Validation	Hold out whole songs, never segments of training songs. Track SDR per stem and listen to difficult passages each round.	Prevents leakage that inflates scores.
Stop rule	Stop when validation SDR no longer improves, or when listening shows new artifacts.	Avoids chasing the metric.

> **On free GPU time**
>
> The free GPU tiers are generous but unpredictable. Kaggle's weekly GPU allowance is widely reported as about 30 hours, but its documentation does not publish a fixed number and it can change; Colab publishes no quota at all. Plan training as short, resumable runs and check your own account's quota before scheduling work.

Run-time performance

Published measurements give a realistic picture of what to expect. Hardware differs a great deal, so you will measure your own laptop first.

Setup	Reported result	Source type
PyTorch, CPU, tested machine	About 19 seconds for 120 seconds of audio (about 6 times faster than real time)	Package benchmark, on a modern Apple laptop
ONNX Runtime, CPU, single fine-tuned specialist	About 22 seconds for a 3-minute song (about 0.20 of real time)	Model card, Apple M4 Pro
ONNX Runtime, CPU, all four specialists	About 88 seconds for a 3-minute song (about 0.49 of real time)	Model card, Apple M4 Pro
TensorRT on a desktop GPU	About 5 seconds per song including file handling	Model card, RTX 3090
Original project guidance for CPU	Roughly one and a half times the track duration	Project documentation, described as optimistic for the fine-tuned models

For planning, assume a typical student laptop CPU is slower than the machines above. A single-model pass may take around the duration of the song, and a four-model bag several times that. That is fine for an offline demo and is another reason to use short clips for experiments and run full songs only for showcase tracks.

Optimisations, in order of payoff

1. **Process once, reuse always.** Cache separation outputs by audio fingerprint and version. Later changes to placement or rendering never repeat the expensive step.
2. **Export to ONNX Runtime.** Reported CPU speed-ups are modest (about 1.3 times over PyTorch in one measurement) but removing the PyTorch dependency also reduces install size.
3. **Half precision on GPU.** Usually a large speed-up with negligible quality change; always confirm with the quality metrics.
4. **Batch segments.** Process several overlapping segments at once on a GPU.
5. **Choose the tier per track.** Use the fast single model by default; use the ensemble only for showcase tracks or when the quick pass produces warnings.
6. **Quantisation and distillation (later).** Smaller, faster student models trained to imitate the large ensemble. They need careful evaluation because they usually lose a little quality.

Measuring success

Question	Measurement	Pass condition
Is stem quality better than the baseline?	SDR and related scores on held-out songs and a hand-picked hard set.	Consistent improvement, not a single lucky song.
Is bleed lower?	Interference score and a listening test on vocal-only and drum-only playback.	Fewer audible leaks in blind comparisons.
Does it help the final mix?	Preference tests on the spatial render using each separator.	Listeners prefer the improved version.
Is anything lost?	Residual level and null test of stems plus residual against the input.	Sums to the original within rounding.
Is it robust?	Run on old mono recordings, live recordings and loud masters.	No crashes, and artifacts are flagged by the guard.

Decision: HT-Demucs as the baseline, a RoFormer model as the second opinion, with licensed fine-tuning as the long-term path	
Choice	Start with HT-Demucs through an inference-only package or ONNX export. Add one RoFormer-class model for comparison. Fine-tune on data you are allowed to use.
Why	Demucs is the best-documented and most practical open baseline, with a permissive code license. RoFormer models show the highest published scores. Combining families reduces artifacts, and licensed fine-tuning solves the legal question while improving quality on your genres.
Drawbacks	Two models double the compute. Weight licenses must be checked before any commercial use. Fine-tuning gains on a small dataset are uncertain.
Alternatives	Use only the newest RoFormer model (higher scores, heavier, unclear weights), use only Demucs (simpler, but weaker on vocals), or train from scratch (needs far more data and compute than is realistic for free).

8 Subsystem 2: analysis and mix intelligence

This subsystem turns audio and stems into a placement plan. It is where the project's originality lives, so it is designed to be explainable, testable and improvable one piece at a time.

Step 1: analysis

Measurement	Method	Library or tool	Used for
Tempo and beats	Onset detection and beat tracking	librosa or madmom	Timing of any movement and reverb length
Key and harmony (optional)	Chroma features	librosa, essentia	Reserved for later features
Loudness	ITU-R BS.1770 integrated and short-term	pyloudnorm or FFmpeg	Level matching and guard
Stereo width	Mid/side energy ratio and inter-channel correlation, per band	NumPy	Consistency with the original image
Dynamics	Crest factor and loudness range	NumPy	Reverb send and limiter behaviour
Spectral balance	Band energies per stem	NumPy	Masking analysis and bass protection
Stem activity	Energy over time per stem	NumPy	Which stem leads at each moment
Recording type	Mono detection, bandwidth, noise floor	NumPy	Gentler profile for old recordings
Style probabilities	A classifier on audio embeddings	Small model, trained on labelled data	Choosing and blending templates

The style classifier

- **Input.** A fixed-size summary of audio features or an embedding from a pretrained audio model, computed over several short windows and averaged.
- **Output.** Soft probabilities over styles, not a single label. A track that is 70% Afrobeats and 20% pop receives a blend of the two templates.
- **Model.** Start with a simple, interpretable model such as logistic regression on embeddings. Move to a small neural network only if it beats the simple model in cross-validation.
- **Data problem.** Public genre datasets under-represent African styles and fusion genres. Build a small, carefully labelled set of your own, and report accuracy per style, not just overall.
- **Honesty rule.** If the top probability is low, the system falls back to a neutral template rather than guessing.

Step 2: the placement policy

The policy is a set of templates plus rules that adapt them to a particular track. Templates live in readable configuration files, so they can be edited, versioned and reviewed like code.

```
template: afrobeats
  anchors: { kick: centre, bass: centre, vocals_lead: centre }
  wide: { perc: spread_pair(az=70), backing_vocals: pair(az=65), keys:
      pair(az=35) }
  surround: { pads: rear_pair(az=150), tails: rear_pair(az=145) }
  movement: { allowed_on: [fx, pads], max_speed: slow }
  room: { rt60: 0.40..0.55, predelay_ms: 10..18 }
  low_end: { mono_below_hz: 120 }
  orig_blend: { vocals_lead: 0.35, drums: 0.40, other: 0.20 }
```

Adapting a template to a track

1. **Blend templates** by the style probabilities.
2. **Scale width with the original.** If the stereo master is already wide, keep the image wide; if it is nearly mono, keep it modest. The plan multiplies spread by a factor derived from the measured width.
3. **Reduce movement when the arrangement is dense.** Dense, fast percussion makes motion tiring. Movement strength is limited by onset density.
4. **Respect the artist.** Apply the intensity cap and protected elements from the consent record last, so they always win.
5. **Choose the room** from the style and the tempo: shorter reverb for fast material, longer for slow.

Masking-aware placement (the original idea)

When two sounds occupy the same frequency range, they mask each other. Spatial separation helps the listener hear both. The policy therefore computes a **spectral overlap** score for each pair of stems and spreads overlapping stems further apart, while keeping anchors fixed.

```
overlap(a, b) = sum over bands of min(E_a(band), E_b(band)) / sum over bands of
    E_a(band)
azimuth gap(a, b) = base_gap * overlap(a, b) * intensity
(anchors keep az = 0; the gaps are applied to the non-anchor stems)
```

This is a design hypothesis, not a proven rule. The plan tests it directly: render with and without masking-aware spreading and compare in blind tests. If listeners do not prefer it, it is removed, which is the right outcome for a hypothesis.

Step 3: from rules to learning

Stage	What is learned	Training signal	Risk
Rules only	Nothing; templates are hand-written.	None	Rigid on unusual tracks.
Learned adjustments	Small offsets to template parameters from track features.	Listening-test preferences.	Needs enough test data; can overfit.
Learned style embedding	A representation of how well-mixed spatial tracks are arranged.	Licensed multichannel or Atmos references, if available.	Data access and licensing.
Preference model	Which of several candidate plans listeners prefer.	Pairwise comparisons from listening tests.	Subjective; needs many listeners.

Each stage must beat the previous one in blind tests before it is adopted. This protects the project from complexity that does not pay off.

Step 4: the fidelity guard

The guard is the safety net that keeps the engine faithful to the record. It runs the checks from the quality report and returns pass, fail or review.

Check	Method	Starting threshold (to tune)
Alignment	Peak of cross-correlation between original and render	Within 1 ms of zero lag
Downmix	Fold render to stereo; compare band energies with original	Within a few decibels per band, to be tuned by listening
Low end	Correlation and level below 120 Hz	High correlation; level within 1 dB
Loudness	Integrated loudness difference	Within 0.5 LU
True peak	Oversampled peak measurement	Below the platform ceiling
Mono sum	Level change when summed to mono	Small loss; no deep nulls
Artifacts	Detect clicks, dropouts, abrupt gain changes, energy jumps at segment joins	No flags, or flags below a severity limit

The thresholds above are starting points. The plan sets the real values by measuring many tracks and listening to the ones near the limit.

Decision: rule-based policy first, with a written, reviewable plan file	
Choice	Hand-written, versioned templates adapted by measured features, producing a plan file for every render.
Why	A rule-based system is explainable, testable and fast to build, and it gives a baseline against which any learned model must prove its value. The plan file supports artist approval and debugging.
Drawbacks	Templates can be rigid. Writing good ones needs real mixing knowledge and many listening sessions.
Alternatives	An end-to-end neural network that outputs the spatial mix directly (less explainable, needs data that is hard to obtain, and riskier for 'do not change the record'), or a learned policy from the start (needs data you do not yet have).

9 Subsystem 3: the spatial renderer

The renderer is the part that must be correct to the last decimal, because errors here are directly audible. It is also the most deterministic part of the system, which makes it the easiest to test thoroughly.

Signal flow in detail

1. **Input.** Stems (and residual), each as float stereo at 44.1 kHz, plus the plan.
2. **Per-stem staging.** Convert each stem to the processing form: mono stems are summed to a mono object; stereo stems that should stay wide are split into a mid object and a side pair, or treated as two objects at mirrored angles.
3. **Direct sound.** Apply gain for distance, then filter with the left and right head-related impulse responses for the stem's direction.
4. **Room sends.** Send a controlled amount of each stem to the room model.
5. **Early reflections.** A few delayed, attenuated, filtered copies computed from a simple room model, each spatialised as arriving from its own direction.
6. **Late reverb.** A dense tail made by a feedback delay network, with decay and brightness set by the template, decorrelated between the ears.
7. **Bass management.** Sum the region below the crossover to centre and keep it mono.
8. **Original blend.** Mix a plan-controlled amount of the unmodified original, time-aligned, back in.
9. **Loudness and limiting.** Match integrated loudness to the original; apply a true-peak limiter.
10. **Output.** Convert to 24-bit with a single dither step and write FLAC.

Head-related filters in practice

- **Format.** Open datasets use the SOFA format. A SOFA file holds an array of impulse responses indexed by source direction.
- **Direction lookup.** Real directions rarely match measured ones exactly. Use the nearest set of measured points and interpolate between them. Interpolating impulse responses naively can cause tonal artifacts, so a common approach is to separate each response into a delay and a minimum-phase filter and interpolate those.
- **Delay handling.** The measured interaural delay is kept as a pure delay, and the common bulk delay is removed. This preserves the interaural cue while keeping the render aligned with the original.
- **Lengths.** Use short filters, a few hundred samples, for direct sound. Long filters are only needed for the room tail, which is handled separately.
- **Headphone compensation.** Some datasets include headphone responses. A later feature can apply a compensation filter for the listener's headphones.
- **Choice of subjects.** Offer two or three profiles. Comparing a few subjects in a listening test is a fast way to learn which suit most listeners.

Dataset	Subjects	Directions	License note
ARI	Over 90 in standard form	About 1,550 per subject	Creative Commons Attribution-ShareAlike 3.0 (requires sharing derived works under the same license)

Dataset	Subjects	Directions	License note
HUTUBS	96	440	Check the dataset page before use
CIPIC	45	1,250	Check the dataset page before use
SADIE II	18	2,818	Check the license in the downloaded files; also provides headphone responses
Viking	A small set	Measured at University of Iceland	Creative Commons Attribution 4.0

The subject and direction counts come from a published comparison of public datasets. The license column is the one to double check before release: a share-alike license, for example, has consequences if you build derived products.

Rooms: early reflections and late reverb

Component	Method	Why	Alternative
Early reflections	Image-source method for a simple rectangular room, first two or three orders	Cheap, controllable, gives the brain a size and distance cue	Measured room impulse responses (realistic, but licensing and data issues)
Late reverb	Feedback delay network with frequency-dependent decay	Smooth tail, adjustable, low cost	Convolution with a measured tail (natural but not adjustable)
Decorrelation	Different delay lines for left and right	Gives a sense of envelopment	All-pass filter chains

Doing it fast enough

An estimate shows that the cost is small. Direct filtering with a 256-tap response at 44.1 kHz needs about 11 million multiply-adds per second of audio per ear, so roughly 22 million per source. With eight sources and a 3.5 minute song that is on the order of 40 billion operations if done naively, which a laptop does in tens of seconds, and far less with fast Fourier transform methods. The reverb is shared, since all stems feed one room, so it adds almost nothing per source. The renderer is therefore not a bottleneck; separation is.

Implementation path and language choice

Phase	Language and tools	Purpose
1	Python with NumPy and SciPy (fast Fourier transform convolution)	Get it correct, with tests, quickly.
2	Same, profiled and vectorised	Find real hot spots instead of guessing.
3	A Rust or C++ core for the hot loops, called from Python	Speed and reuse; the same core can compile to WebAssembly for the browser.
4 (later)	Real-time version for the phone, using the same core	Head tracking and on-device rendering.

Decision: write the renderer's own DSP instead of depending on an audio effects library	
Choice	Implement convolution, delay lines and the reverb network directly, using SciPy first. Treat open effects libraries as references and test oracles.
Why	The core of Stage is the renderer, and owning it avoids license entanglements: Spotify's open Pedalboard library is available under the GPL v3, which has obligations that may not suit a proprietary product. Writing the DSP also demonstrates real engineering depth.
Drawbacks	More work and more risk of bugs. Mitigation: test against known results, such as an impulse response and sine sweeps, and compare with a library on identical settings.
Alternatives	Use Pedalboard (fast to start, GPL obligations), Steam Audio or Resonance Audio (open spatial audio toolkits, geared to games and real-time use; check their licenses and fit), or a commercial plugin (not free).

Decision: object-style rendering with direct filtering rather than ambisonics	
Choice	Each stem is rendered as its own source with its own direction, filtered by head-related responses.
Why	Accuracy and clarity: you can place, test and explain each stem. It matches the plan file exactly. For offline work, the extra cost over ambisonics is negligible.
Drawbacks	Head tracking later needs either re-filtering in real time or an ambisonic intermediate.
Alternatives	Render to ambisonics, then decode to binaural: smooth for head turns and scales well with many sources, but at limited order the localisation is blurrier. It is a good candidate for the real-time generation.

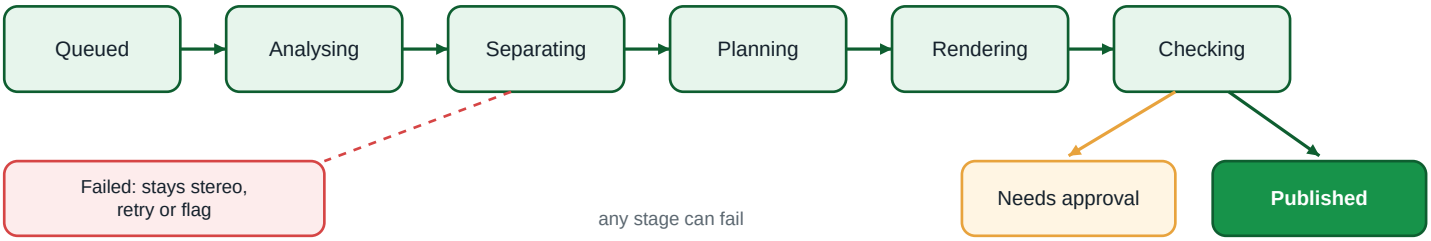


Figure 3. The life of one job. Failure at any stage leaves the track in normal stereo.

10 Back end: orchestration, services and reliability

The demo needs almost no back end, because it is a set of files and a static page. This chapter describes the platform-scale back end anyway, because a reader at Spotify will want to see that the design holds up beyond a laptop. Each part is small, and each one has a demo equivalent.

Components

Component	Responsibility	Demo equivalent
Trigger	Starts a job when a master is accepted and the consent check passes.	A command that processes a folder.
Deduplicator	Looks up the audio fingerprint; reuses stems and renders if the same master was already processed with the same versions.	A cache folder keyed by fingerprint.
Orchestrator	Runs the stage graph, tracks state, retries, and records timings and costs.	A simple Python loop with a state file.
GPU workers	Run separation. Scale with queue depth.	Your laptop or a free notebook.
CPU workers	Run analysis, planning, rendering and the guard.	Your laptop.
Consent service	Stores and serves opt-in, caps, protected elements and approval.	A YAML file.
Approval service	Shows renders to the rights holder and records the decision.	A manual step in the demo.
Metadata writer	Publishes the spatial block on the track record only after pass and approval.	A JSON file read by the web player.
Observability	Logs, metrics and alerts; cost per track.	A summary report.

Job lifecycle and idempotency

Every job is **idempotent**: running it twice with the same inputs and versions produces the same outputs and never publishes twice. This one rule removes a whole class of production problems, because retries become safe.

- **Keys.** Each stage output is stored under a key made from the audio fingerprint and the version tuple. If the key exists, the stage is skipped.
- **Atomic publish.** The spatial block on the track record is written in one step, after all checks. A reader never sees half a result.
- **Retries.** Transient failures, such as a worker being pre-empted, are retried with increasing delay. After a few failures the job is flagged for review and the track stays in stereo.
- **Poison inputs.** A master that crashes a stage repeatedly is quarantined so it does not block the queue.
- **Versioning.** A new engine version creates new keys and does not overwrite old results. The old version stays live until the new one is approved.

- **Withdrawal.** When a rights holder withdraws consent, the metadata block is removed first, then the assets are deleted by a background job.

Interfaces

Endpoint or event	Direction	Payload	Purpose
ingest.accepted (event)	In	track id, fingerprint, master location	Starts processing.
GET consent/{track}	Out	opted_in, modes_allowed, caps, protected, approval	Policy for the job.
POST spatial/{track}/assets	Out	mode, asset id, loudness, version tuple	Registers a finished render.
POST spatial/{track}/approval	In	decision, approver, note	Records the artist's decision.
DELETE spatial/{track}	In	reason	Withdraws spatial assets.
GET spatial/{track}	Out	status, modes, asset ids	What the client reads.
job.status (event)	Out	stage, timing, cost, verdict	Monitoring.

Failure modes and how they are handled

Failure	Effect if unhandled	Handling
Worker pre-empted mid-job	Lost work	Checkpoint after each stage; retry on a new worker.
Separation produces heavy artifacts	Poor spatial mix	Guard flags it; fall back to a single-model pass, or leave in stereo.
Master is mono or very narrow	Odd spatial image	Detect early; use the gentle profile.
Silence or corrupt audio	Crash or meaningless output	Input validation stage rejects it with a clear reason.
Guard threshold too strict	Most tracks rejected	Monitor rejection rate; thresholds are configuration, changed by review.
Release-day surge	Queue grows, deadline missed	Priority classes and autoscaling workers.
Storage or network error	Partial writes	Write to temporary keys and rename on success.
Model or HRTF update	Silent change in sound	Version tuple; old renders stay until the new ones are approved.

Capacity and the one-to-two-hour goal

A simple model shows that the time target is achievable with modest hardware, because the work is parallel by track. Suppose a surge of 10,000 opted-in tracks arrives in an hour and each needs 60 seconds of GPU time for separation.

```
GPU time needed = 10,000 tracks * 60 s = 600,000 s = about 167 GPU-hours
To finish in 1 hour : about 167 GPUs working at once
To finish in 2 hours : about 84 GPUs
Cost of the surge : 167 GPU-hours * $0.71 per hour = about $118 at one cloud's
    listed on-demand L4 price
                    167 GPU-hours * $0.28 per hour = about $47 at a listed spot
                        price
```

The GPU prices come from public price lists and vary by provider and region. Even a large surge is cheap in money terms, so the real design problem is availability and scheduling, not cost. A sensible time budget for one track is: queue wait up to 30 minutes at the 95th percentile, separation a few minutes, analysis, planning and rendering a few minutes more, and the guard under a minute. Human approval runs on its own clock and does not count against the target.

Security and privacy

- **Least privilege.** Workers can read masters and write only to their own output area.
- **Unreleased music.** Masters can be unreleased, so access is logged, and assets are encrypted at rest as a default of the cloud platform.
- **No training on the catalog without rights.** Fine-tuning uses licensed data only. Catalog audio is processed, not learned from, unless the agreement says so.
- **Auditability.** The plan, the version tuple and the consent record are stored with every render, so any output can be explained and reproduced.
- **Demo data.** The public demo uses only your own or licensed music and has no user accounts or personal data.

Decision: a batch graph with idempotent stages rather than a long-running service per track	
Choice	Stages are pure functions of their inputs, executed by an orchestrator that records state and timings.
Why	Simple to reason about, easy to retry, easy to scale by adding workers, and a close match to how Spotify already processes audio (Klio on Apache Beam). Pure stages are also the easiest to test.
Drawbacks	Less suited to interactive, real-time use. It also needs good state tracking so work is not repeated.
Alternatives	A microservice per stage with synchronous calls (more moving parts and more failure modes), or a serverless function per stage (simple but awkward for GPU work and large files).

11 Front end and user experience

The front end does two jobs. It is the product demonstration, showing what a listener would experience. It is also the research instrument, collecting blind listening-test results. Both must work on a modest phone and a weak connection.

Platform choices

Option	Strengths	Weaknesses	Verdict
Web app (static page)	Shareable by link, no install, free hosting, works on every phone and laptop.	Browser audio quirks; background playback limits on some phones.	Primary demo.
Android app	Closest to the real product; background playback; can mark streams as already spatialised.	Needs install; harder for others to try; more work.	Optional second build.
iOS app	Large audience.	Needs paid developer program and Apple hardware for builds.	Out of scope for a free project.
Desktop app	Full control.	Least shareable.	Not needed.

Web stack

Part	Choice	Why	Alternatives
Language	TypeScript	Catches errors before they reach users; good editor support.	Plain JavaScript.
Build tool	Vite	Fast, simple, small output.	Webpack, Parcel.
UI framework	Svelte	Very small bundles, simple state handling, quick to learn for a solo build.	React (largest ecosystem, larger bundle), vanilla web components (smallest, more manual work).
Audio	Web Audio API with the standard media element for long tracks	Native, free, and gives sample-accurate control for short clips.	Howler.js (convenient, less control), Tone.js (powerful, larger).
Waveform display	A small custom canvas or a light library	Keeps size down.	wavesurfer.js (feature rich, heavier).
Styling	Plain CSS with design tokens and a dark theme	Light and fully under your control.	Tailwind (fast to write, a build dependency).
Storage of test answers	Local browser storage, then a small free form or serverless endpoint	No server to run.	Your own server (more to maintain).
Hosting	A free static hosting service with HTTPS	Free, global, simple deploys from Git.	Check each host's size and bandwidth limits for audio.

The audio engine inside the page

The most delicate part of the front end is switching between stereo and spatial without a gap or a jump. There are two situations, and each has its own solution.

Situation	Approach	Reason
Short clips (15 to 30 seconds) for A/B tests	Decode each version into memory and play both from the same start time through two gain nodes, crossfading in a few milliseconds when switching.	Sample-accurate alignment and no clicks. Both versions are always in sync.
Full tracks in the player	Two media elements, kept in sync by periodically correcting the hidden one's position, with a short crossfade on switch.	Streams large files without decoding everything. Small timing drift is corrected in the background.

- **Alignment depends on the engine.** Because renders are aligned within about a millisecond of the original, switching is seamless. If the renderer ever introduces delay, the A/B switch will expose it immediately, which makes the front end a useful test of the engine.
- **Loudness matching.** A/B comparisons must be level-matched, or louder always sounds better. The page applies a stored gain offset so both versions have the same loudness.
- **Blind mode.** In the listening test, the labels A and B are randomly assigned each time and the true order is saved with the answer.
- **Format support.** Modern browsers play FLAC; test on the phones and browsers your audience uses and keep a high-bitrate fallback for any that fail.

Screens

Three screens cover the whole demo. They are wireframed below and described in detail.

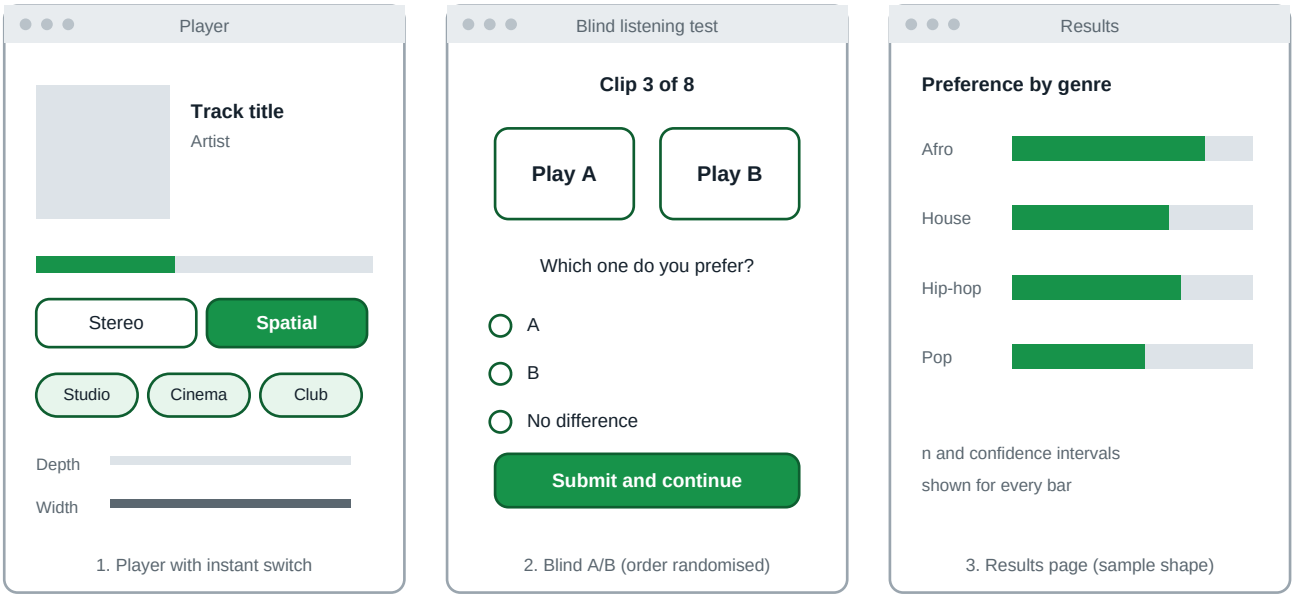


Figure 4. Wireframes: the player, the blind listening test, and the results page.

Player

- Track title, artist, artwork placeholder, and progress bar.
- A two-state switch: Stereo or Spatial. The active state is obvious in colour and in text, so it never depends on colour alone.
- Mode chips: Studio Room, Cinema, Club, Open Air. Each has a one-line description on hover or focus.
- Depth and width sliders, with a reset button.

- A small information panel on 'What changed in this mix?', which reads the plan file and shows it in plain words, such as 'Lead vocal centred, percussion widened, pads placed behind you'.
- A visual of the sound field: a simple top-down view showing where each stem sits, taken from the plan. It makes the idea understandable at a glance.

Blind listening test

- A short introduction asking people to use headphones and set a comfortable volume.
- Six to eight clips, each with two versions labelled only A and B in random order.
- One question per clip: which do you prefer, with a 'no difference' option and an optional comment.
- A progress indicator and the ability to stop at any point; partial results still count.
- A final page with a few questions about listening habits, then thanks. No personal data is collected.

Results page

- Preference rates by genre and by mode, always shown with the number of responses and a confidence interval.
- Plain-language reading of what the numbers mean and how sure we are.
- A download link for the anonymous data, so others can check the analysis.

Designing for data and low-end phones

- **Nothing heavy loads until asked.** Audio is fetched only when the user taps play.
- **Short clips.** A 15-second lossless clip is on the order of 1.5 megabytes, so a test of six clips with two versions each is roughly 18 megabytes. A data-saver option offers high-bitrate compressed versions of both A and B at a fraction of that size, with the limitation reported in the results.
- **Small code.** Keep the initial page small, so it loads quickly on a weak connection.
- **Offline friendly.** Cache already-fetched clips so a refresh does not repeat the download.
- **Works on small screens.** Controls sized for thumbs, one-column layout.

Accessibility

- Every control is reachable by keyboard and labelled for screen readers; the switch announces its state.
- Colour contrast follows common web accessibility guidance, with text contrast of at least 4.5 to 1.
- No reliance on colour alone, and no flashing content.
- Keyboard shortcuts for the test: one key per version, one for submit.
- A visible warning that very loud or very wide effects can be uncomfortable, with the width slider as a remedy.

Words that the interface uses

Element	Wording	Reason
Toggle	Stereo | Spatial	Short and clear; no brand jargon.
Mode: Studio Room	Close, warm, like a mix room	Describes the feeling, not the technology.
Mode: Cinema	Wide and big, like a screening room	Plain description.
Mode: Club	Punchy, controlled low end	Tells what changes.
Mode: Open Air	Natural and airy	Short and visual.
Unavailable	The artist has not enabled spatial mixing for this track.	Explains why, and respects the artist.
Credit line	Automated spatial mix	Honest labelling; avoids implying a human engineer made it.

Android build (optional)

- Kotlin with the standard media library (Media3), which plays FLAC and handles background playback and notification controls.
- Set the audio attributes of the stream to say it is already spatialised, so a phone's own spatializer, if present, does not process it again.
- Read the same manifest and files as the web demo, so there is one source of truth.
- Use the app to demonstrate that the renders play on a real phone with no Dolby support enabled, which is the central claim of the project.

Decision: a static web demo as the primary showcase, with an optional Android build	
Choice	TypeScript and Svelte on Vite, Web Audio for clips, media elements for full tracks, hosted as static files.
Why	Anyone can try it from a link, which is the most important property of a pitch demo. It is free to host, and the same page doubles as the listening-test instrument. The code is small and fast.
Drawbacks	Browser audio has quirks across devices, and background playback on phones is limited. It does not prove behaviour inside a native app.
Alternatives	A native Android app only (closest to the real product but much harder to share), or a framework such as Flutter or React Native (one code base for both phones, heavier tooling and less direct audio control).

12 The compute question

This is the question you are most likely to be asked: *a model that processes every song sounds expensive; how do you keep the compute bill under control?* This chapter builds the answer from numbers, shows which levers matter most, and ends with a short version you can say out loud.

Where the compute actually goes

Stage	Hardware	Relative cost	Why
Analysis	CPU	Low	Signal measurements and a small classifier.
Separation	GPU (preferred)	High, dominant	A large neural network runs over the whole song.
Mix planning	CPU	Very low	Rules and small models.
Rendering, per mode	CPU	Low to medium	Short filters, one shared reverb; scales with modes.
Guard	CPU	Low	A handful of measurements.

The consequence is simple: reducing compute means reducing how often, and how heavily, separation runs. Everything else is small.

A cost model with real prices

Public GPU price lists show that an NVIDIA L4, a low-power inference GPU, is listed at about $0.71 per hour on-demand on one major cloud, with spot prices in the range of $0.26 to $0.32 per hour, and $0.44 to $0.80 per hour from other providers. Spot instances can be interrupted, which is acceptable for a batch job that checkpoints. The table below turns those prices into a cost per track and per million tracks for several assumptions about how long separation takes.

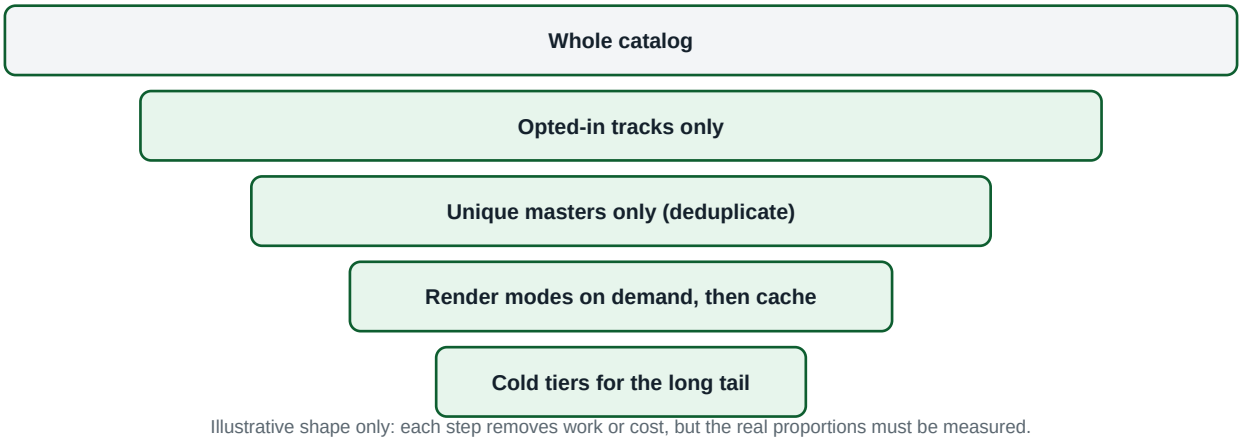
GPU seconds per track	On-demand ($0.71 per hour): per track	per million tracks	Spot ($0.28 per hour): per track	per million tracks
10	$0.0020	about $1,970	$0.0008	about $780
30	$0.0059	about $5,920	$0.0023	about $2,330
60	$0.0118	about $11,830	$0.0047	about $4,670
120	$0.0237	about $23,670	$0.0093	about $9,330

The assumptions are labelled in the header. The real GPU time per track must be measured. Published benchmarks suggest a single separation pass on a desktop GPU takes seconds, so 10 to 60 seconds per track is a reasonable planning range, including an ensemble and overhead.

> **The key fact**
>
> Processing a million tracks costs in the low thousands to low tens of thousands of dollars of GPU time at public list prices. Compute is a one-time cost per track, because the result is stored and reused. This is very different from a cost that recurs on every play.

The levers, from most to least powerful

1. **Process only what is opted in.** The consent design is also the biggest cost control. Only tracks whose rights holders agreed are ever processed.
2. **Process each master once.** The same recording often appears on an album, a single and a compilation. Matching by audio fingerprint means it is separated once and the stems are reused.
3. **Separate once, render many.** Stems are cached. Adding a mode, changing a template or updating the renderer never repeats separation.
4. **Render modes on demand, then cache.** Rather than rendering every mode for every track in advance, render the default mode eagerly and the others when first requested.
5. **Cascade by quality tier.** Use a fast single model by default and the heavier ensemble only for tracks where the guard flags problems, or for high-profile releases.
6. **Use the cheapest suitable hardware.** L4-class GPUs for inference, spot instances for batch work, and CPU for everything that does not need a GPU.
7. **Optimise the model.** Half precision, ONNX or TensorRT export, batching. Reported speed-ups are substantial on GPU.
8. **Distil and quantise (later).** Train a smaller model to imitate the ensemble. This lowers cost further but needs careful quality checks.



Illustrative shape only: each step removes work or cost, but the real proportions must be measured.

Figure 5. Each lever removes work. The widths are illustrative, not measured.

Worked example: the cascade

Suppose a fast pass takes 10 GPU seconds and an ensemble takes 60. If 20 percent of tracks need the ensemble, the average is 0.8 times 10 plus 0.2 times 60, which is 20 GPU seconds, one third of the cost of running the ensemble on everything. If separate cache hits from duplicates remove another quarter of the work, the effective average falls to about 15 seconds. These percentages are assumptions to be measured, but the structure of the saving is real.

Why not just make the model lightweight?

A smaller model is one lever, not the whole answer, and it carries a quality cost. Separation quality is the foundation of the product, so the plan keeps the strongest model where it matters and spends the savings elsewhere. The order is: remove unneeded work first, reuse work second, choose cheaper hardware third, and only then shrink the model, and only if listening tests show no loss.

Comparison with the cost of the alternative

Any honest answer also compares against what the alternative costs. A Dolby Atmos route requires a new mix for each track, which is human studio work and is far more expensive per track than a few GPU seconds. It also requires the playback side to support the format. I have not verified the price of any Dolby licensing arrangement, so the plan makes no claim about it; the claim it can support is that automated processing has a measured, small marginal cost per track.

The short answer

> **What to say when asked**
>
> Compute is paid once per track, not once per play. We only process tracks that rights holders opt in, we process each unique master once, and we reuse the separated stems for every mode. On public GPU prices the separation step costs a fraction of a cent to a couple of cents per track, and we have levers to cut it further without touching quality: caching, a fast-first cascade, spot GPUs and optimised inference. We will publish the measured cost per track from the demo.

Decision: cache stems and render modes lazily, with a fast-first cascade for separation	
Choice	Store stems by fingerprint and version. Render the default mode eagerly and other modes on demand. Use a single-model pass first and the ensemble only when needed.
Why	It attacks the dominant cost (separation) directly and turns every later change, such as a new mode or a better template, into a cheap CPU job.
Drawbacks	More complex logic and a cache to manage. Stems take storage space (Chapter 13). On-demand rendering adds a delay for the first listener to ask for a mode.
Alternatives	Render every mode eagerly (simplest, highest storage and compute), or render only at play time on the device (no storage, but needs the real-time renderer and phone compute).

13 The storage question

The second likely question: *you are saving extra versions of every song; how do you keep storage from exploding?* This chapter sizes the data, prices it with public numbers, and shows how to keep it under control.

Size of one track

```
Raw 24-bit, 44.1 kHz stereo = 44,100 * 24 * 2 = about 2.12 megabits per second =
    about 15.9 MB per minute
FLAC compression saves a variable amount; for planning, assume about 9 to 11 MB per
    minute
A 3.5 minute song is therefore about 32 to 38 MB; this document uses 35 MB as a
    round figure (an estimate, to be measured)
```

What gets stored

Item	Size per track (estimate)	Needed for	Tier
Original master	About 35 MB	Already stored by the platform	Existing
Stems plus residual (for example 5 files)	About 5 times 35 MB, so about 175 MB	Re-rendering new modes and later generations	Cold
Rendered modes (each)	About 35 MB	Streaming to listeners	Hot or warm
Plan and report	Kilobytes	Approval, audit, debugging	Warm

The stems are the largest item. They are also the most valuable to keep, because they make every future improvement a cheap CPU job instead of a new separation run. The question is whether keeping them is worth their storage cost, which depends on the storage tier.

Public storage prices

Tier (regional, list price)	Per GB per month	Retrieval fee	Minimum time	Use for
Standard	About $0.020	None	None	Renders being streamed
Nearline	About $0.010	About $0.01 per GB	30 days	Renders rarely played; recent stems
Coldline	About $0.004	About $0.02 per GB	90 days	Stems kept for re-rendering
Archive	About $0.0012	About $0.05 per GB	365 days	Long-term retention

These are published list prices for one major cloud in 2026 and vary by region and by location type (a few sources list Standard in some US regions at slightly above $0.020). A company the size of Spotify negotiates its own rates, so the figures show the order of magnitude, not the real bill.

What it costs at scale

The table shows yearly storage cost for one rendered mode at list prices, for different catalog sizes. A round figure of 35 MB per track is used.

Tracks with a render	Total size	Standard per month	Standard per year	Nearline per year
1 million	35 TB	about $700	about $8,400	about $4,200
10 million	350 TB	about $7,000	about $84,000	about $42,000
100 million	3.5 PB	about $70,000	about $840,000	about $420,000

For the stems at 175 MB per track, the same arithmetic applies five times over. At 100 million tracks that is 17.5 petabytes. On a cold tier at $0.004 per GB-month, that is about $70,000 per month, or about $840,000 per year; on Archive at $0.0012 it is under a third of that.

> **Putting it in proportion**
>
> Even the most expensive reading of these numbers, with the entire catalog, at list prices, is small compared with Spotify's revenue. The point is not that storage is free; it is that storage is not the real constraint. The real constraints are quality, rights and engineering effort. The cost still has to be controlled, because efficiency is a sign of good engineering and because opt-in covers only part of the catalog anyway.

For scale: Spotify reported about 4.5 billion euros of Premium revenue in the third quarter of 2026 alone. Storing a full render of a 100-million-track catalog at list prices costs a fraction of one percent of one year of that revenue. That is a statement about proportion, not a claim that Spotify would approve the spend.

The levers

1. **Opt-in only.** Only a share of the catalog is ever processed. Store nothing for the rest.
2. **Deduplicate by fingerprint.** A master used on several releases is stored once.
3. **Do not store what is cheap to recompute.** Renders in rarely used modes can be deleted and recreated from stems on demand.
4. **Tier by demand.** A small fraction of tracks gets most plays. Keep renders of popular tracks hot, and move the long tail to cheaper tiers, accepting a small delay on first play.
5. **Keep stems cold.** Stems are only read when something changes, so a cold tier suits them. Respect minimum storage periods and retrieval fees when modelling this.
6. **Reuse the encoding ladder.** A lossless render does not have to be the only version; lower-bitrate versions made by the existing encoder serve listeners on limited data.
7. **Lifecycle rules.** Automatically move or delete assets based on age and use.
8. **Option B in the long run.** Store stems and placement data once and render on the device, so the number of stored versions stops growing with the number of modes.

Streaming bandwidth is a separate question

Storing a render and sending it to a listener are different costs. A spatial render streamed as lossless has the same size per minute as any other lossless track, so for a listener who already streams lossless, the per-play delivery cost does not change. The effect on delivery cost depends on how many listeners switch from lossy to lossless to use spatial audio, which is another reason to offer a lossy spatial version through the existing encoder. This is a question for the demo's measurement plan, not something to assume.

The short answer

> **What to say when asked**
>
> We do not store spatial versions for the whole catalog. We store them only for opted-in tracks, once per unique master, and keep the large separated stems on a cold tier. At list prices a full lossless render of a hundred-million-track catalog is in the hundreds of thousands of dollars per year, small against the revenue of the service, and each lever we have cuts that further. In the long run we can store stems and placement data once and render on the device, which stops storage growing with the number of modes.

Decision: keep stems on a cold tier and renders on warm or hot tiers, with lifecycle rules	
Choice	Stems are written once and rarely read. Renders live where they are streamed, and move to cheaper tiers when demand falls.
Why	It puts the largest data (stems) on the cheapest storage and the data that must be read quickly on the fastest. It also keeps the option to improve the product without new separation work.
Drawbacks	Retrieval fees and minimum storage periods apply to cold tiers. A re-render triggered after a long wait has a small delay and cost.
Alternatives	Delete stems after rendering (cheapest storage, but every upgrade repeats the expensive separation), or keep everything hot (simplest, most expensive).

14 Testing and quality assurance

Audio software needs two kinds of testing: the ordinary kind that checks code, and a measurement kind that checks sound. Both are automated as far as possible, and the rest is done by careful listening.

The test pyramid for Stage

Level	What it tests	Examples	How often
Unit tests	Small functions with known answers.	The interaural delay formula at 90 degrees gives about 0.66 ms; level drops about 6 dB per doubling of distance; loudness of a calibrated tone matches its known value.	Every change.
Property tests	Rules that must always hold.	Output length equals input length; silence in gives silence out; stems plus residual sum to the input; output never exceeds the true-peak ceiling.	Every change.
Golden-file tests	Exact or near-exact comparison with a stored result.	A fixed 10-second clip with a fixed plan renders to a stored file within a tiny tolerance.	Every change.
Signal tests	Behaviour with synthetic inputs.	An impulse placed at 90 degrees produces a delayed, quieter response in the far ear; a sweep shows the expected filtering; correlated noise reveals phase errors.	Every change.
Integration tests	The stages working together.	A short real clip goes through the whole pipeline and the guard passes.	Daily or per merge.
Robustness tests	Bad inputs.	Silence, mono files, 16-bit files, a different sample rate, a truncated file, extreme loudness.	Per merge.
Performance tests	Speed and memory.	Seconds of processing per minute of audio at each stage.	Weekly and before release.
Listening tests	How it sounds to people.	Blind A/B and ABX with a panel.	Per major change.

Testing the renderer against its own plan

The renderer can be verified without any listening. Place a test signal at a chosen direction using the plan, then measure what arrives at each ear and compare it with what the plan promised: the interaural time difference, the level difference, and the direct-to-reverberant ratio. This closes the loop between the plan and the sound, and catches sign errors such as left and right being swapped, which are easy to make and very easy to miss by ear.

Regression of results, not only code

- Keep a fixed set of showcase tracks and clips. After every significant change, regenerate the quality reports and compare them with the last accepted ones.
- Fail the check if a metric moves by more than an agreed tolerance without an explanation.
- Render a short comparison page with before and after spectrograms, so a reviewer can see what changed at a glance.

Continuous integration

- Run formatting, linting, unit, property and golden-file tests automatically on every push, using a free CI service on the hosting platform (check the current free allowance).
- Run the slower integration and performance tests on a schedule or on demand.
- Run model-heavy jobs on your own machine or free GPU notebooks, never in CI.
- Build the web demo in CI and fail the build if the bundle grows beyond an agreed size.

Listening tests in detail

Test	Question it answers	Method	Notes
Blind A/B preference	Which do people prefer?	Two versions in random order, level matched, with a no-difference option.	The main pitch evidence.
ABX	Can people hear any difference at all?	A, B and an unknown X; identify which one X matches.	Useful for fidelity and for lossy-versus-lossless questions.
MUSHRA-style rating	How good is each version on a scale?	Several versions, a hidden reference and a low-quality anchor, each rated.	Good for comparing separators or templates.
Task test	Can people locate and follow sounds better?	Ask where a sound appears; ask them to follow the backing vocal.	Checks that spatial placement conveys something.
Comfort check	Is it tiring or uncomfortable?	A short question after longer listening.	Important for acceptance.

Protocol

1. Fix the question and the success criterion before running the test.
2. Match loudness across versions with the same meter.
3. Randomise the order and the A/B assignment; store the assignment with each answer.
4. Ask people to use headphones and record the type they used.
5. Include repeated trials and a few obviously different pairs as attention checks.
6. Report the number of participants, the number of trials, and the method, even when the result is unflattering.

Statistics you will actually use

Situation	Method	How to say it
Proportion who prefer Stage	A confidence interval for a proportion (the Wilson interval behaves well for small samples).	'62 percent preferred the spatial version (95 percent interval 44 to 77, 29 trials).'
Is it better than chance?	Binomial test against 50 percent.	'22 of 30 choosing Stage corresponds to a p-value of about 0.016.'
Compare two versions on a rating scale	Paired test on each listener's difference.	'Version B scored higher by 0.8 points on average.'
Many listeners, repeated clips	Treat the listener as a random effect, or summarise per listener first.	Avoids counting one person's answers as independent.
Several comparisons at once	Adjust for multiple comparisons or declare the main question in advance.	Prevents finding patterns that are just noise.

With around 100 participants, the margin on a proportion is about plus or minus 10 percentage points; with 30 it is about plus or minus 18. Say so in the report. A small honest study is more credible than a larger one with exaggerated claims.

Device checks

- Play the renders on at least two phones, two pairs of wired headphones and two pairs of Bluetooth headphones.
- Test on a phone with a system spatializer switched on and off, to confirm the 'already spatialised' marking works.
- Note Bluetooth codec limits and any audible differences, and include them in the limitations section.

15 The demo and how to show it

The demo has to do three things in under two minutes: show that it works on an ordinary phone, let a person hear the difference, and make the design visible. Everything else supports those three goals.

Scope, in priority order

Priority	Feature	Why
Must	Six to eight showcase tracks across genres and eras, each with a stereo version and at least three spatial modes.	The evidence that the engine works across styles.
Must	The player with an instant Stereo/Spatial switch and mode chips.	The core experience.
Must	The blind listening test and the results page.	Evidence, and a way for visitors to take part.
Must	A 'what changed' panel and the top-down sound-field view.	Makes the idea understandable without listening.
Should	A comparison of separators on difficult passages with the metrics.	Shows the separation work.
Should	A simulated artist dashboard from Figure 5 of the first document.	Shows the control model.
Should	A cost and timing panel: measured seconds and estimated cost per track.	Answers the efficiency question with data.
Could	An Android build.	Shows it on a real device.
Could	A lossy-version comparison to test the encoding-ladder idea.	Answers an open design question.

Choosing the showcase material

- Use only music you own, have written permission to use, or that carries an open license. Your own productions are ideal, especially if you hold the project files and so also have true stems to test the separator against.
- Cover several styles. For example: an Afrobeats-style track, a house-style track, a hip-hop track, a pop or R&B track, and a band recording.
- Include one hard case on purpose, such as a dense mix or an old mono recording, and show how the engine handles it. Honest handling of a difficult case builds more trust than a flawless set.
- Use 15 to 30 second clips for the listening test, and a few full songs for the player.

> **Why your own productions matter**
>
> Tracks you own with project files are doubly valuable: you can compare the separator's output with the true stems, which is a rare and persuasive result. It is the strongest way to show that the separation work is real and not just a good-sounding demo.

The two-minute video

Seconds	Scene	On screen	Narration idea
0 to 10	The gap	A phone with a music app; a label showing spatial audio unavailable	Many listeners cannot hear spatial audio, because it depends on the phone and on a special mix.
10 to 25	The idea	The Stage player on an ordinary phone	This engine runs inside the app and needs no special phone hardware.
25 to 60	The sound	Switching between Stereo and Spatial and between modes, with the sound-field view	Listen to the same song, first in stereo, then around you.
60 to 80	The control	The artist dashboard	Artists opt in, preview and approve.
80 to 100	The evidence	Results chart with sample size and confidence intervals	Here is what listeners preferred, and how sure we are.
100 to 120	The ask	Contact and link to the write-up	This is an independent proposal. Here is the full technical plan.

The landing page

1. A one-sentence promise and a play button above the fold.
2. The player with the stereo and spatial switch.
3. 'How it works' in five short steps with the pipeline diagram.
4. The listening test invitation.
5. The results, with honest limitations.
6. Links to the technical write-up and the code.
7. A clear note that the project is independent and not affiliated with any streaming service.

What to publish

Item	Contents	Audience
One-page summary	Problem, idea, evidence, link.	Busy readers.
Technical write-up	Architecture, methods, measurements, limitations (these documents are the base).	Engineers.
Public repository	Code, schemas, configs, test manifests, a clear README.	Engineers and reviewers.
Data and scripts	Anonymous listening-test data and the analysis notebook.	Anyone who wants to check.
Demo link and video	The experience.	Everyone.

Questions to prepare for

The answers below are short versions. Each is backed by a chapter in this document or in the first one.

Question	Short answer	Where it is covered
Why not just license Dolby Atmos?	A licensed format still needs a new mix per track and support in the playback device. Owning the renderer works on every phone and is under Spotify's control. I have not claimed anything about licensing prices.	First document, Chapter 2; this document, Chapter 12
Is it as good as a hand-made Atmos mix?	No, and it does not claim to be. It reaches the large part of the catalog that will never get such a mix. The gap is measured and reported.	First document, Chapter 16
Stereo-only sources cannot be spatialised properly.	Separation recovers the parts; placement is designed to hide separation artifacts; a share of the original always stays in the mix.	Chapters 7 and 8
What about compute cost?	Paid once per track; a fraction of a cent to a couple of cents in GPU time at list prices; many levers to reduce it.	Chapter 12
What about storage?	Opt-in only, deduplicated, stems on a cold tier; a full render of a hundred-million-track catalog is hundreds of thousands of dollars a year at list prices.	Chapter 13
Will artists accept it?	It is opt-in with preview and approval, labelled as an automated mix, and easy to withdraw.	First document, Chapter 11
Does it alter the record?	A built-in guard compares each render with the original for level, low end, tone and alignment, and blocks anything that drifts.	Chapter 8
What about head tracking?	A later generation; the stems-plus-data design supports it. Android already has a standard spatializer interface, but it depends on each phone maker supplying the effect engine.	Chapters 3 and 9
How do you know people want it?	Existing surveys suggest quality matters to many listeners, but one recent study found high-fidelity audio ranked lower than it used to and bonus features ranked higher among people considering a switch. The demo's own listening test and survey collect direct evidence.	Chapter 14
Can the models be used commercially?	The demo uses open baselines. For a product, weights need a clear license or must be replaced by models trained on licensed data, which is part of the plan.	Chapters 7 and 16

16 Licensing, rights and responsible use

The research found that licensing is where an otherwise sound technical plan can fail quietly. Every component has a license, and licenses differ in what they allow. This chapter lists what was verified, what is unclear, and how the plan reduces the risk. It describes product design, not legal advice.

Components and their license posture

Component	What was found	Implication	Action
Demucs code	MIT license.	Free to use and modify, with the copyright notice.	Keep the notice; use a maintained fork.
Demucs pretrained weights	Original notes described them as for research use, citing dataset terms; some mirrors tag them MIT; a public question about this was open.	Unclear for commercial use.	Use for the demo; confirm with the maintainers; plan licensed fine-tuning.
RoFormer weights	Several community weights list the license as unknown or record none.	Cannot be assumed free to use commercially.	Use for comparison only; do not ship them.
Training datasets (MUSDB18-HQ and others)	Generally released for research use.	Training a product model on them may not be allowed.	Read each license; prefer data you own or have permission to use.
Klio	Apache license; built by Spotify for audio pipelines.	Permissive.	Use as a reference design; do not depend on it.
Pedalboard	GPL v3, built by Spotify's Audio Intelligence Lab.	Copyleft: it can require that software linking to it is also released under the GPL.	Do not link it into the renderer; use only as a comparison tool.
ARI HRTF data	Creative Commons Attribution-ShareAlike 3.0.	Attribution required; derived data must carry the same license.	Attribute; keep derived HRTF files under the same terms.
Viking HRTF data	Creative Commons Attribution 4.0.	Attribution required.	Attribute.
SADIE virtual loudspeaker data	Free to use under the Apache license with attribution of the original dataset.	Permissive with attribution.	Attribute.
Other HRTF sets	Not verified in this research.	Unknown.	Read each dataset page before use.

Content rights

- **Test music.** Use music you own or that is licensed for your use. Public posts that include other people's recordings can be removed, and can damage trust in the project.
- **Derived audio.** A spatial render of a recording is a modified version. In a platform setting this requires the signed agreement described in the first document.
- **Training data.** Do not train on catalog or commercial music without clear rights. The plan fine-tunes on licensed material only.

- **Disclosure.** Label spatial versions as automated mixes, and say so in the demo.

Names and brands

- Use original mode names, or get permission before using a studio's name. Studio names are trademarks.
- Refer to Dolby, Dolby Atmos, Apple Music and Spotify only to describe facts, and say clearly that the project is independent and not affiliated or endorsed.
- Do not copy Spotify's logo, trade dress or exact interface. The mockups are illustrations of an idea.

Privacy and safety

- The public demo collects no personal data. Listening-test answers are anonymous, with no names, emails or device identifiers.
- Tell participants what is collected and why, and allow them to stop at any time.
- Warn about loud volume and strong effects, and offer a way to reduce them.

Decision: treat any pretrained weight as research-only until its license is confirmed	
Choice	Use pretrained models for the demo and comparisons, but design the product path around weights whose license is clear or trained on licensed data.
Why	It lets the demo reach high quality quickly without making a legal claim it cannot support. It also turns a weakness into a strength: licensed fine-tuning is a real piece of engineering a company would value.
Drawbacks	Licensed data is hard to get and fine-tuning gains on small data are uncertain.
Alternatives	Ignore the question (fast, but dangerous for any commercial claim), or train from scratch (clean license, far more data and compute than is realistic).

17 Risks and open questions

A plan is credible when it names what could go wrong. The register below lists the main risks, the early warning sign for each, and what to do about it.

Risk	Likelihood	Impact	Early warning	Response
Separation artifacts audible in spatial mix	High	High	Listeners notice glitches on certain stems	Ensemble, cleanup, more original blend, place artifact-prone stems toward diffuse areas.
Spatial version not preferred over stereo	Medium	High	Listening test result near 50 percent	Rework templates; try subtler intensity; report honestly. The research result is still valuable.
Generic HRTF fits few listeners	High	Medium	Reports of in-head or front-back confusion	Offer several profiles; add a quick selector.
Model weight license blocks commercial claims	Medium	High	Maintainers or license text rule it out	Fine-tune on licensed data or use a clearly licensed model.
Laptop too slow for the work	High	Medium	Minutes per clip for experiments	Use short clips; free GPU notebooks; cache outputs; process only showcase tracks in full.
Free GPU quota changes or is unavailable	Medium	Medium	Quota not shown or sessions cut short	Short resumable runs; fall back to CPU; reduce experiments.
Mobile data runs out	Medium	Medium	Downloads stall	Pre-download once on a good connection; keep a local cache and lockfiles.
Participants do not complete the listening test	High	Medium	Few responses after sharing	Make it short; let partial answers count; ask people directly; recruit through the chapter and campus groups.
Over-ambitious scope	High	High	Stages slip; nothing is finished	Each stage ends with something that works; cut features, not quality.
Spotify does not respond	High	Medium	No reply to outreach	Treat the project as a strong portfolio piece; share publicly; improve based on feedback.
Behaviour differs on real phones	Medium	Medium	Different sound on different devices	Test on several devices; document limitations.
Bluetooth lossy codecs change the effect	Medium	Medium	Spatial cues fade over Bluetooth	Test and report; recommend wired for critical listening.

Open questions and how to settle each

Question	How to settle it	When
Can the pretrained Demucs weights be used commercially?	Ask the maintainers; read the license of each weight file; plan the fine-tuning route regardless.	Before any commercial claim.
How fast is separation on my laptop?	Benchmark one clip with each model on CPU, record seconds per minute of audio.	Stage 0.
Which HRTF subjects suit most listeners?	Render a test set with three subjects and compare in a listening test.	Stage 1.
Does the ensemble beat the best single model?	Compare SDR and listening results on a held-out hard set.	Stage 2.
Does masking-aware placement help?	Blind A/B with and without it.	Stage 3.
Do style templates beat one fixed template?	Blind A/B on several styles.	Stage 3.
Does lossy compression damage the spatial cues?	ABX between lossless and high-bitrate lossy versions of the same render.	Stage 6.
Do listeners want spatial audio enough to change behaviour?	The survey and the listening test; compare with existing studies.	Stage 6.
Is Klio still the right reference for Spotify's pipelines?	Check the project's current status and recent activity.	Before the write-up.
How much does spatial audio add to delivery cost?	Model lossless versus lossy mix for the target users.	Platform write-up.

18 The build sequence

The stages from the first document are repeated here with concrete tasks, entry conditions and exit conditions. Each stage ends with something that works and can be shown, so progress is never lost.

Stage	Tasks	Exit condition
0 Foundations	Install tools once on a good connection. Create the repository and layout. Build the test library manifest. Write the loudness and metrics tools. Benchmark separation speed on your laptop. Set up the listening-test skeleton.	Any clip can go through a baseline separation and a metrics report, and the numbers are written down.
1 Renderer	Load SOFA data. Implement direct filtering with interpolation, the delay handling, distance gain, a first room model, bass management and the limiter. Add signal tests and the plan-versus-sound test.	A test sound placed at chosen directions measures correctly at the ears and renders to FLAC.
2 Separation lab	Wrap the baseline model. Add overlapping-segment processing, the residual, a second model and the combination step. Add cleanup for the 'other' stem. Start fine-tuning on licensed data.	Measured improvement over the baseline on held-out tracks, confirmed by listening.
3 Mix intelligence	Build the analysis stage, the style classifier, the template files, the placement policy and the guard. Write the plan files and the quality reports.	Showcase tracks across styles render with passing reports and sound good.
4 Modes	Define each mode as configuration. Tune rooms and levels. Add the first car-related experiments as research.	Each mode is clearly different and passes the guard.
5 Listener app	Build the player, the A/B switch, the mode chips, the sound-field view and the 'what changed' panel. Optional Android build.	A stranger can understand the feature within a few seconds on a phone.
6 Evaluation	Run the blind listening tests and the survey. Run the objective metrics across the library. Fix what the results reveal.	A results report that states what works and what does not.
7 Release and pitch	Record the video, publish the page, the write-up and the data. Prepare the outreach message.	A package that can be understood and tried in minutes.

The first ten working sessions

A concrete start removes the hardest part, which is beginning. Each item is a single working session.

1. Create the repository, the folder layout and the first decision records.
2. Install the Python environment and the audio tools; download the models and one HRTF set once, on a good connection.
3. Write the audio input and output module and the loudness meter; test them on calibrated tones.
4. Run the baseline separation on one short clip and time it; write the number in the log.
5. Compute the stems-plus-residual null test and fix any mismatch.
6. Load a SOFA file and render one test sound at a chosen direction; measure the delay and level at each ear.
7. Add distance gain, bass management and the limiter; test with property tests.
8. Write the plan file schema and render a hand-written plan for a real clip.
9. Listen critically to the result against the original; write down what is wrong, specifically.
10. Fix the biggest problem, re-render, and record the before and after.

Definition of done for the demo

- Six showcase tracks in at least three modes, all passing the guard.
- A public page with the player, the blind test and the results.
- Measured cost and timing per track, published.
- A written list of limitations, as honest as this document.
- A repository that a stranger can clone and reproduce a result from.

Keeping the work sustainable

- Work in small stages with something playable at the end of each one.
- Protect your study time; the plan has no calendar dates on purpose, so it can bend around exams.
- Keep the project public and documented as you go. Notes written along the way become the write-up.
- Ask for feedback early, from friends who produce music and from audio engineers. Their ears are the fastest bug detector.

19 Version 2: what changed and why

After the first specification was written, the four areas where the project could fall short were researched in depth: the **sound** of spatialised stereo, **separation quality** against the state of the art with very little compute, the **scope**, and **Spotify's reaction**. This part of the document (Chapters 19 to 26) is the result. It adds new methods. It does not remove anything from Chapters 1 to 18.

> **How the two halves fit together**
>
> **Precedence rule.** Where a v2 chapter and an earlier chapter describe the same thing differently, the v2 chapter wins, because it was written with more evidence. The mapping table below shows exactly where that happens. Everything else in the earlier chapters stays valid.

An honest statement about guarantees

No design can guarantee that listeners will love the result, that a model will beat every benchmark, or that a company will reply. What a design can do is turn each risk into a **testable hypothesis** with a pass or fail gate and a fallback that still gives a good result. That is what v2 does. A risk that has a measurement and a fallback is no longer a place where the project can fail silently.

The four weak spots and the v2 answer

Weak spot	What the research found	The v2 answer	Chapter
Sound of spatialised stereo	Filtering with head-related responses alone usually gives poor externalisation; rooms and reflections are needed. Mixers using Atmos give each element its own virtual distance, and some complain when a renderer treats everything the same.	**Pan-Anchored Spatial Rendering**: preserve where each sound already sits in the stereo image, widen it with an order-preserving warp, give each role its own distance, keep timbre within a measured bound, and treat ambience separately.	21
Separation quality and compute	Newer models reach similar quality with far less compute (SCNet reports 9.0 dB average SDR on MUSDB18-HQ at 48 percent of HT-Demucs CPU time). SDR also does not capture what matters after spatial placement.	**Triage-and-refine** (heavy models only where a cheap proxy says they are needed), **render-aware fine-tuning** (train on the rendered output, not just the stems), then distillation and careful quantisation.	22
Scope too big	The risk is not the idea but the number of unfinished parts.	**Three complete tiers** (Lite, Standard, Studio), five vertical slices, gates and kill criteria.	20 and 26
Spotify may not notice or respond	Spotify's terms treat ideas sent to it as non-confidential and usable without payment; its developer terms bar using Spotify content to train models.	Make the work public, dated and easy to evaluate; never use Spotify content; build credibility with open results and a live demo.	25

Mapping to the earlier plan

Earlier item	What v2 does to it	Status
Chapter 7: separation pipeline	Adds triage-and-refine, render-aware fine-tuning, distillation and quantisation. The baseline and licensing guidance stay.	Extended
Chapter 8: mix intelligence	Adds the pan-anchored placement rules and the timbre bound. Templates and the fidelity guard stay.	Extended
Chapter 9: renderer	The renderer becomes the PASR core, written in Rust and runnable in real time. Direct per-object rendering stays as the offline reference for tests.	Replaced by an extension
Chapter 14: testing	Adds hypothesis gates and new metrics (Chapter 26).	Extended
Chapter 15: demo	Adds the drop-any-song page and the X-ray view (Chapter 24).	Extended
Chapter 18: build sequence	Reordered around five vertical slices so something works in the first week (Chapter 20).	Reordered
First document: three modes and artist control	Unchanged. Modes are still configuration; artist control still applies to Standard and Studio.	Unchanged

How to read Part II without getting lost

1. Read Chapter 20 first. It gives the whole project on one page and the three tiers.
2. Read Chapter 21 if you care about the sound, and Chapter 22 if you care about the models. They are independent.
3. Read Chapter 23 before writing any code: it fixes the languages and the size and speed budgets.
4. Chapter 24 is the demo; Chapter 25 is the plan for reaching Spotify; Chapter 26 is the list of experiments that decide what is kept.

20 Scope made simple: one page, three tiers, five slices

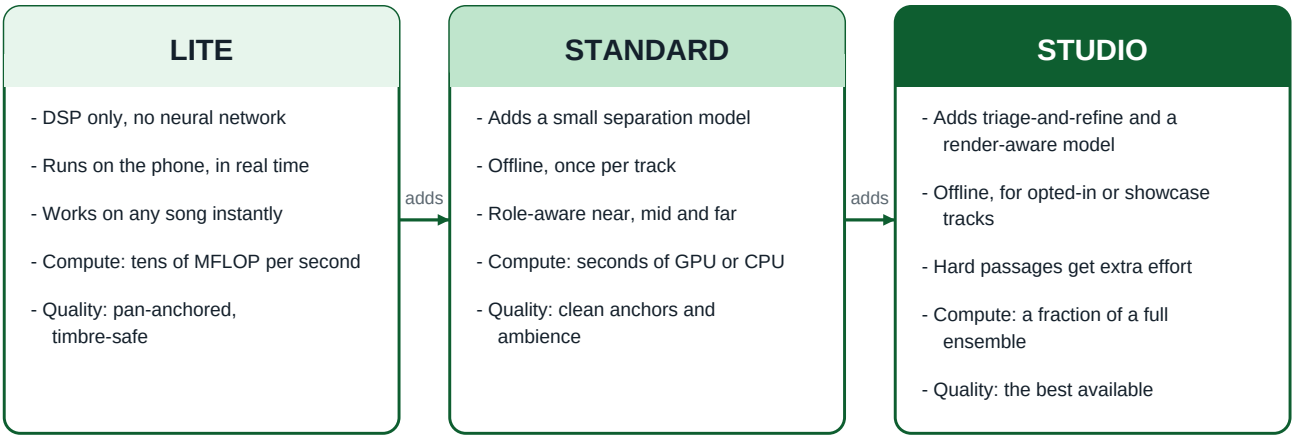
The project is large, so the first job is to make it small enough to hold in your head without making it smaller in ambition. The tool for that is structure: a one-page description, three tiers that are each complete, and five vertical slices that always leave something working.

Stage on one page

Question	Answer
What is it?	A spatial audio engine that lives inside the music app, so it works on any phone and any song, with the artist in control.
How does it work?	It keeps each sound where the stereo mix already puts it, spreads those positions around the listener, gives each role the right distance, and adds a believable room. Machine learning separates the parts when more precision is needed.
What is new?	Position-preserving widening, role-based distance, a measured timbre bound, a guard that protects the original, and separation trained on the rendered result.
What runs where?	Lite runs on the phone in real time. Standard and Studio run once per track on a server or a laptop.
What does it cost?	Lite: a small fraction of one CPU core. Standard and Studio: seconds of compute per track, paid once.
How do we know it is good?	Blind listening tests, objective checks against the original, and published measurements of cost and speed.
What is the demo?	A web page where you drop any song and hear it around you, plus showcase tracks with the best quality and a view that shows what the engine is doing.

Three tiers, each a complete product

A common way to shrink a project is to cut features until a weak minimal version remains. That is not what this does. Instead, the same product is built three times, from lightest to richest. Every tier uses the same plan file, the same guard and the same interface, and each is good enough to show on its own. Moving up a tier adds quality and costs compute; moving down never breaks anything.



Same plan file, same fidelity guard and same interface in every tier. Each tier is a complete product.

Figure 6. The three tiers. Lite needs no neural network at all, which is what makes it portable.

	Lite	Standard	Studio
Needs a neural network	No	Yes, small	Yes, small plus a refiner
Where it runs	On the listener's device, in real time	Offline, once per track	Offline, once per track
Works on any file instantly	Yes	No (needs processing time)	No
Uses the separated stems	No	Yes	Yes, with cleaner stems
Role-aware distance	By frequency and position only	Yes, by role	Yes, by role
Artist opt-in needed	Applies only to the listener's own playback; a platform rollout would still need rights decisions	Yes	Yes
Best for	Demo reach, portability, a fallback for everything	The main product path	Showcase tracks and flagship releases

Why Lite matters

Lite is the most important design decision in v2. A version that needs no model is portable to any phone and browser, costs almost nothing to run, makes the demo work on any song someone drops in, and acts as the fallback whenever the heavier tiers fail. It also gives a clean baseline: every later improvement has to beat Lite in a blind test to earn its cost.

The ladder of baselines

Every improvement must be compared with something honest. The ladder below fixes the comparisons in advance, so progress is measured, not felt.

Level	What it is	Must beat
B0	Plain stereo, level matched	Nothing; it is the reference

Level	What it is	Must beat
B1	A generic stereo virtualiser: two virtual speakers at 30 degrees with a small room (what a device effect roughly does)	B0 in preference
B2 (Lite)	Pan-Anchored Spatial Rendering on the mix, no stems	B1 in preference
B3 (Standard)	PASR on stems with role-based distance	B2 in preference
B4 (Studio)	B3 with the refined, render-aware separation	B3 in preference or at lower cost

Five vertical slices

A vertical slice is a thin cut through every layer of the project that works from end to end. The plan is built slice by slice, so that after each one there is a complete, demonstrable result. Nothing waits for a later slice to be useful.

	1 Hear it	2 Prove it	3 Make it smart	4 Make it light	5 Show it
Renderer	Lite in Python on one clip	Timbre and externalisation	Roles: near, mid, far	Rust core and WASM	Real time in the browser
Separation	Baseline run and timing	Score on MUSDB test	Triage and refine	Quantise and distil	Efficiency chart
Mix plan	One fixed template	Plan file and guard	Style templates, masking	Per-track tuning	Artist controls (mock)
App	Files and headphones	A/B page	Mode chips	Drop any song page	X-ray view and polish
Evidence	Your own ears and notes	Blind test v1 (10 people)	Blind test v2 (30 people)	Performance budgets	Write-up and video

Figure 7. The work breakdown. Read each column top to bottom as one slice; read each row left to right as one track growing richer.

Slice	Goal in one sentence	Done when
1 Hear it	One clip, spatialised by Lite in Python, heard on headphones.	You can switch between stereo and spatial and describe the difference honestly.
2 Prove it	Measurements and a first blind test show it is better than a generic virtualiser.	Ten listeners, a result with an interval, and the guard passes.
3 Make it smart	Stems, roles, style templates and masking-aware placement raise quality.	Standard beats Lite in a blind test with thirty listeners.
4 Make it light	The core is rewritten in Rust, runs in real time in the browser, and meets the size and speed budgets.	Drop any file and hear it live; the budgets are met.
5 Show it	The demo, the efficiency chart, the write-up and the video.	A stranger understands it in two minutes.

Estimated effort, in working sessions

A working session is one focused sitting of a few hours. These are planning estimates, not promises, and you should replace them with your own after the first slice. The point of listing them is to see the shape of the work.

Slice	Rough sessions	Main risk	Fallback if it overruns
1	8 to 12	Setup and getting the maths right	Reduce to one clip and one HRTF subject.
2	8 to 12	Recruiting listeners	Smaller panel, report wider intervals.
3	20 to 30	Separation compute and quality	Ship Lite plus Standard with the fast model only.
4	15 to 25	Learning Rust and WebAssembly audio	Keep the Python core for the demo; do the Rust port later.
5	8 to 12	Polish and recording	Cut visuals, keep the audio and the measurements.

Now, next, later: nothing dropped, only sequenced

Now (slices 1 and 2)	Next (slices 3 and 4)	Later (after slice 5)
Lite renderer, timbre bound, fidelity guard, A/B page, first blind test.	Stems and roles, style templates, triage-and-refine, Rust core, WebAssembly, drop-any-song page.	Render-aware fine-tuning at scale, distillation, car and speaker rendering, head tracking, Android build, artist dashboard with a real back end.

Rules that keep the scope under control

1. **Always have a working version.** Never start a new slice while the last one is broken.
2. **One new idea at a time.** Add one technique, measure it, keep it or remove it.
3. **Decide the test before the work.** Every feature has a hypothesis and a gate (Chapter 26).
4. **Time-box experiments.** An experiment that has not shown a gain in a fixed number of sessions is parked, with notes.
5. **Write it down.** Each session ends with three lines: what was done, what was measured, what is next. These become the write-up.

21 The sound: Pan-Anchored Spatial Rendering

This chapter is the core of the v2 design. It starts from what the research says makes spatialised stereo sound bad, derives a set of rules that avoid those problems, then turns the rules into mathematics and a signal flow that is light enough to run on a phone. Everything here is labelled as established practice or as a hypothesis to be tested.

21.1 Why spatialised stereo usually sounds bad

Problem	Evidence found	Consequence for the design
Sound stays inside the head	Virtualisers that rely only on head-related filters usually do not achieve good externalisation or perceived distance; room reverberation and reflections must be added to the direct sound.	Every source gets reflections and a late tail; the room is not optional.
Tone changes	Studies of binaural music rendering rate timbre as a separate attribute, next to externalisation and preference, and Atmos mixers report that mid and far binaural settings change the low end.	A measured timbre bound, and a protected low end.
Everything gets the same room	A mixer on Apple's developer forum reports that Apple's headphone re-render does not follow the per-object near, mid and far settings, so intimate elements get the same ambience as spacious ones. This is one user's observation, not a confirmed specification.	Distance is set per role, so a lead vocal can stay dry and close while pads sit far away.
Front and back confusion	A known limitation of generic head-related filters.	Important elements stay in front; the rear is used for ambience.
Separation errors become audible	Moving a badly separated part exposes its artifacts.	Positions come from the original image, and a share of the original always stays in.
Gimmick effect	Mixing guidance for Atmos music says to keep kick, snare, bass and lead vocal front and centre and use the rest of the space for supporting elements.	Anchors stay put; width goes to elements that tolerate it.

> **Supporting evidence**
>
> Research on headphone rendering found that primary-ambient extraction and source separation can improve externalisation and enlarge the sound stage with minimal coloration. This is the scientific basis for building on the stereo image instead of inventing positions.

21.2 The principle: change position, not the record

Five rules follow. Each one is a constraint that can be checked automatically.

Rule	Statement	How it is checked
P1 Pan fidelity	A sound that is left of another in the original stays left of it. Widening is order-preserving.	Rank correlation of positions before and after equals 1.

Rule	Statement	How it is checked
P2 Timbre fidelity	The long-term spectrum of the render stays within a small bound of the original, band by band.	Timbre deviation (21.6) at most 1.5 dB average, no band beyond 4 dB.
P3 Anchor dryness	Kick, bass and the lead vocal use the near setting and little reflected energy.	Direct-to-reverberant ratio per role.
P4 Ambience is ambience	Diffuse parts of the mix are spread and decorrelated; direct parts keep a clear direction.	Energy split p + a equals the input.
P5 Low end is mono	Below about 120 Hz the render stays centred and in phase with the original.	Correlation and level checks in the guard.

21.3 The signal model

Take the short-time Fourier transform of the stereo input, X_L(t,f) and X_R(t,f), for time frame t and frequency bin f. The model used in stereo upmixing research says each time-frequency bin is a mix of one dominant **primary** sound that is amplitude-panned and an **ambient** part that is uncorrelated between the channels:

```
X = p * [1, k]^T + a
p : primary component (a scalar per bin)
k : primary panning factor, k = |X_R| / |X_L| for a single dominant source
a : ambient component, uncorrelated between the two channels
```

The least-squares estimate of the primary component, which is the standard result in this literature, is:

```
p_hat = (X_L + k * X_R) / (1 + k^2)
primary vector = p_hat * [1, k]
ambient a = X - p_hat * [1, k]
primary + ambient = X exactly: nothing is lost, nothing is invented
```

Position from the level ratio

The panning factor k maps to a position index in the range minus one to plus one, and then to a perceived angle with the tangent law for loudspeakers placed at plus and minus 30 degrees:

```
phi = (k - 1) / (k + 1) (phi = -1 hard left, 0 centre, +1 hard
    right)
theta = atan( phi * tan(30 deg) ) (perceived stereo angle)
Example: k = 3 -> phi = 0.5 -> theta = atan(0.5 * 0.5774) = about 16.1 degrees
    to the right
```

Confidence from coherence

Not every bin has one clear source. A recursive average of the cross-correlation gives a coherence score between 0 and 1:

```
Gamma(t,f) = | E[X_L * conj(X_R)] | / sqrt( E[|X_L|^2] * E[|X_R|^2] )
Gamma near 1: one dominant panned source (use the primary path)
Gamma near 0: diffuse or spaced-microphone content (send to the ambient path)
```

Bins with low coherence are treated as ambience even if the level ratio looks panned. This protects live and spaced-microphone recordings, where level-ratio analysis is unreliable, and it is the main reason the method degrades gracefully instead of producing artifacts.

21.4 Sectors and the widen warp

Rendering every time-frequency bin with its own filter would be expensive. Instead the stereo angle is divided into 13 soft sectors, one every 5 degrees from minus 30 to plus 30. Each bin is shared among nearby sectors with weights that add up to one:

```
w_j(t,f) = exp( -(theta - theta_j)^2 / (2 * sigma^2) ) / sum over j of exp( ... )
theta_j = 5 * j degrees, j = -6 ... +6, sigma = about 2.5 degrees
sum over j of w_j = 1 (energy is conserved)
```

Each sector is then moved to a new azimuth by a monotone widening warp. The warp is a saturating curve, so it widens near the centre and cannot run past the sides:

```
alpha_j = A * tanh( beta * theta_j / A )
A = 80 degrees (largest allowed azimuth), beta = widening factor, 1.0 to 2.4
Example, beta = 2.4: theta = 30 -> alpha = about 57 degrees; theta = 15 -> about
    34; theta = 5 -> about 12
The curve is strictly increasing, so the order of sounds is preserved (rule P1).
```

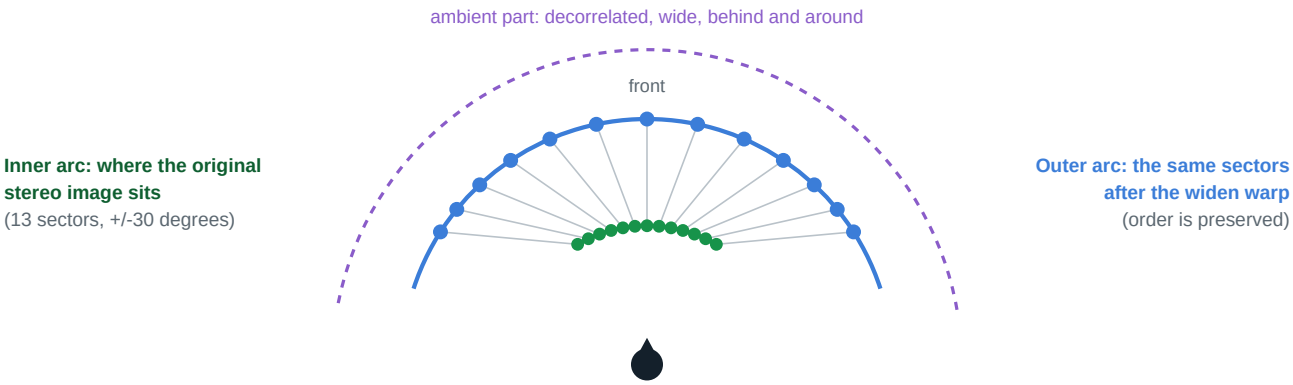


Figure 8. Thirteen sectors of the original stereo image (inner arc) moved to wider azimuths (outer arc). Ambience is handled separately.

21.5 Distance, reflections and the near, mid and far idea

Atmos mixers set each element to near, mid or far to control how much room and distance processing it receives. Stage uses the same idea, made continuous and set automatically from the role. The physics gives a way to choose the numbers. The direct sound falls about 6 dB for every doubling of distance, while the reverberant level stays almost constant, so the direct-to-reverberant ratio (DRR) falls with distance. The distance at which the two are equal is the **critical distance**:

```
DRR(d) = DRR(d0) - 20 * log10(d / d0) (6 dB per doubling)
d_c = 0.057 * sqrt( Q * V / T60 ) (metres; Q = 1 for an omni source,
    V = room volume in m^3)
Example: V = 100 m^3, T60 = 0.45 s -> d_c = 0.057 * sqrt(100 / 0.45) = about 0.85
    m
```

Role	Virtual distance	Target DRR (starting values)	Reflections	Note
Kick, bass	Near, or bypass	High (very dry)	None or minimal	Protects punch and the low end.
Lead vocal	Near	About +6 dB	Light early reflections	Keeps intimacy.
Snare, claps	Near	About +4 dB	Light	Keeps impact.
Hats, shakers, percussion	Mid	About 0 dB	Moderate	Width and detail.
Keys, guitars	Mid	About 0 dB	Moderate	Support.
Backing vocals	Mid to far	About -3 dB	More	Blend.
Pads, ambience, effects	Far	About -6 dB or lower	Full early reflections and tail	Envelopment.

The DRR numbers are starting values, chosen from the physical relation above and from mixing practice, and they are tuned by listening (hypothesis H3). Reflections are not computed for each sector. They are computed on **three shared buses** (near, mid, far) using a short shoebox-room model, so the cost does not grow with the number of sectors.

21.6 Keeping the tone: the timbre bound

Head-related filtering inevitably changes the spectrum. Some of that change is the cue that tells the brain about direction, so it must not be removed entirely. The long-term average, however, can be compensated, which is the idea behind diffuse-field style equalisation. The rule is to match the long-term spectrum and leave the short-term cues alone:

```
E_in(b) = long-term power of the original (mono sum) in ERB band b (median over
    about 3 s)
E_out(b) = long-term power of the render, average of left and right, in band b
G(b) = sqrt( E_in(b) / E_out(b) ), limited to plus or minus 4 dB and smoothed
    across bands
Timbre deviation TD = mean over bands of | 10 * log10( E_out(b) / E_in(b) ) |
    target: at most 1.5 dB
```

Because the compensation is slow (seconds) and limited, it leaves the fast directional cues untouched. Whether it helps is hypothesis H2.

21.7 The ambient path

- The ambient part a is not discarded and not copied: it feeds a late-reverberation tail, with two independent decorrelated tails for the left and right ear.
- The tail uses sparse velvet-noise convolution or a partitioned fast Fourier transform convolution with a shaped noise tail. These are cheap and have a long record in the research literature.

- Several binaural-virtualiser structures that use feedback delay networks are covered by patents held by large audio companies. The plan therefore avoids copying that structure and treats a freedom-to-operate check as a to-do before any product claim. This is not legal advice.
- Rear azimuths are used only for ambience, which keeps important elements in front and reduces front-back confusion.

21.8 Masking-aware placement

When two parts occupy the same frequencies, spreading them apart can make both easier to hear. The placement step picks sector shifts for the non-anchor stems by minimising a simple cost:

```
J = sum over pairs (i, j) of O_ij * exp( -|alpha_i - alpha_j| / D ) + lambda *
    sum over i of ( alpha_i - alpha_i0 )^2
O_ij : spectral overlap of stems i and j (0 to 1) D : about 20 degrees (a tuning
    constant)
alpha_i0 : position from the original image lambda : keeps positions close
    to the original
Anchors are fixed. With at most eight stems this is solved by a few passes of
    coordinate descent, which costs almost nothing.
```

The first term pushes overlapping stems apart; the second keeps them near where the original put them. The constants are starting values and the whole idea is hypothesis H4.

21.9 The signal flow

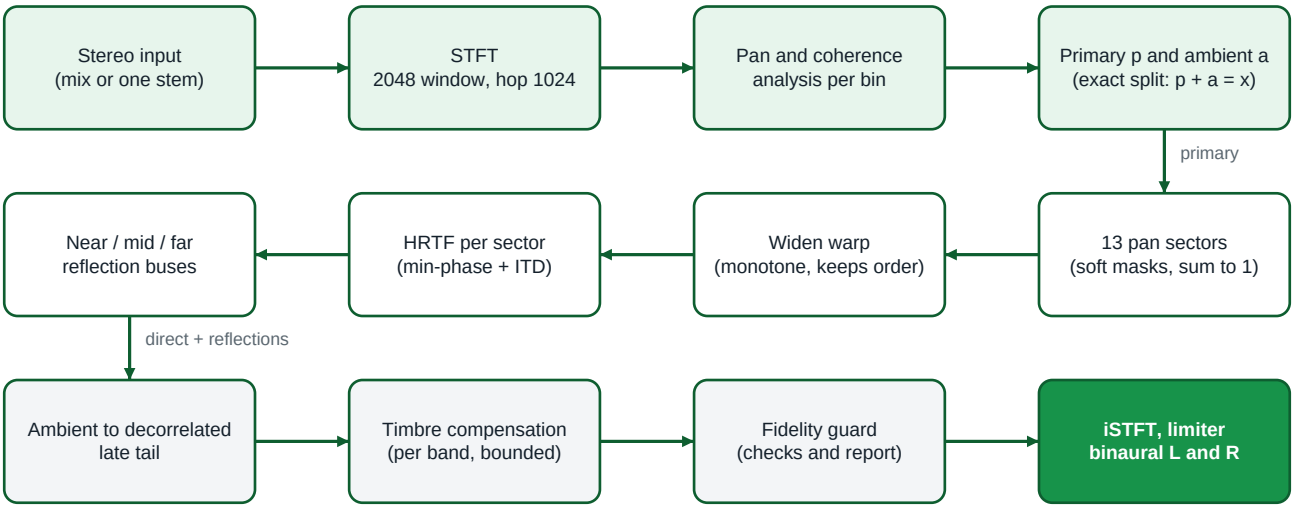


Figure 9. Pan-Anchored Spatial Rendering. The same flow serves Lite (on the whole mix) and Standard and Studio (on each stem).

21.10 The three tiers in terms of this flow

Step	Lite	Standard and Studio
Input to the analysis	The whole stereo mix	Each stem (and the residual) separately
Roles	Guessed from frequency and position: low and centred means anchor, wide and high means supporting	Known from the stem: vocals, drums, bass, other
Distance	Frequency-dependent default	From the role table
Ambient path	Same	Same, from each stem's ambient part
Timbre bound and guard	Same	Same

21.11 Cost and portability

The renderer is light enough to run in real time on a phone. Counting one stereo stream at 44.1 kHz with a 2048-point window and a hop of 1024 samples, there are about 43 frames per second:

```
FFT work : 4 transforms per frame (two forward, two inverse) * about 5 * N *
    log2(N) = 5 * 2048 * 11 = about 113,000 flops each
            4 * 113,000 * 43 = about 19 million flops per second
Sector work: 13 sectors * 1025 bins * 2 ears * about 8 flops = about 213,000 flops
    per frame * 43 = about 9 million flops per second
Reflections and late tail (shared buses): about 25 million flops per second
Total : about 55 million flops per second, on the order of 0.05 to 0.1 GFLOP/s,
    per stereo stream
```

- Memory: 13 sectors * 2 ears * 1025 complex bins * 8 bytes = about 213 KB of filter tables.
- Latency: one window, about 46 ms, which is fine for playback and within the usual 150 ms target for head-tracked audio.
- This is a small fraction of one modern phone core; the exact figure is measured in slice 4 (hypothesis H8).
- For Standard and Studio, the same flow runs offline on each stem, so cost grows with the number of stems but stays small next to separation.

21.12 Prior art and what is new

Honesty about novelty protects credibility. The following ideas are established and are used here with credit: panning-index and ambience-index extraction (Avendano and Jot), azimuth discrimination and resynthesis (Barry and colleagues), primary-ambient extraction and the least-squares estimator (He, Tan, Gan and others), stereo position estimation and mid-side decomposition (Kraft and Zolzer), and per-object near, mid and far binaural distance in Dolby's renderer.

What Stage contributes is the **combination**: order-preserving widening of the original image, role-based distance driven by machine-learned stems, a measured timbre bound, a fidelity guard that protects the record, a tier structure whose lightest member needs no neural network, and separation trained on the rendered result (Chapter 22). Each of these is a testable claim, and none is presented as a proven improvement until the experiments in Chapter 26 say so.

Decision: build the renderer on the original stereo image rather than on template positions	
Choice	Estimate where each sound already sits, preserve that order, widen it smoothly, and add role-based distance and a measured ambient path.
Why	It keeps the mix true to the record, uses decades of stereo-analysis research, works without any neural network in Lite, and makes errors in separation less audible because positions never jump to unrelated places.
Drawbacks	Level-ratio analysis is unreliable for time-panned or spaced-microphone recordings (handled by the coherence gate), and old mono recordings have no image to preserve (they receive a gentle ambient-only treatment).
Alternatives	Template placement from the first document (explainable, but invents positions), or a neural network that outputs the binaural mix directly (less controllable, needs data that is hard to obtain, and weaker on the do-not-change-the-record requirement).

22 Separation v2: better results with far less compute

The goal is not to win a leaderboard on one benchmark with a laptop. It is a more useful target: the **best separation for spatial rendering per unit of compute**, with a licence you control. This chapter defines what 'better' means, surveys what is verified, and designs a pipeline that spends heavy compute only where it changes what listeners hear.

22.1 What beating the state of the art can honestly mean

Claim	Realistic?	How it would be shown
A new record SDR on the standard benchmark with a free laptop	Unlikely. Leading results come from large models and ensembles trained with extra data.	Not a goal.
Equal quality at a fraction of the compute	Plausible, and valuable for a platform.	Efficiency frontier plot: quality against compute.
Lower leakage after spatial placement than a same-cost baseline	Plausible: this metric is not what the standard models optimise.	Render-weighted leakage score (22.8) and blind tests.
A clearly licensed model suited to your genres	Achievable through fine-tuning on data you may use.	Documented data and licences.
A real-time capable fast tier	Plausible with the lightweight model families.	Measured real-time factor.

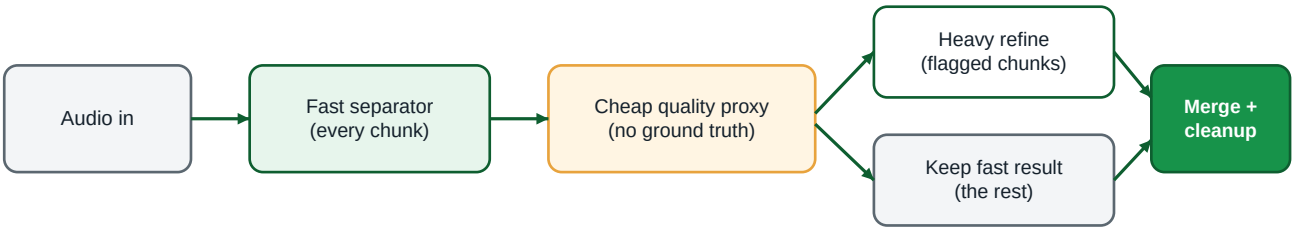
22.2 The verified efficiency landscape

Model or technique	Verified result	Meaning for Stage
HT-Demucs	Strong open baseline. On one benchmark the fine-tuned version scores about 12.05 dB on bass, 11.24 on drums, 8.33 on vocals and 5.74 on 'other'.	Baseline and fallback.
SCNet	9.0 dB average SDR on MUSDB18-HQ without extra data, with CPU inference time 48 percent of HT-Demucs. The paper's single-thread CPU test gives a real-time factor of 1.38 for HT-Demucs and about 0.66 for the selected SCNet; the larger version reaches 9.69 dB. Official code is public.	The best candidate for the fast tier.
Band-Split and Mel-Band RoFormer	Highest published scores for single models (a Band-Split RoFormer reached 11.99 dB average with 500 extra songs).	Candidate refiner for hard chunks.
Real-time single-path TFC-TDF networks	A 2025 paper presents a lightweight real-time low-latency separator and studies quantisation.	Candidate for a future on-device tier.
Quantisation of separators	An IEEE study finds quantisation noise matters mainly when the model produces high SDR, and that the input and output activations are especially sensitive.	Quantise the middle, keep the ends in higher precision.
Weight compression	The original Demucs paper reports that DiffQ quantisation reduced the model from about 1014 MB to about 120 MB with no loss in SDR.	Smaller downloads are possible.
Beyond SDR	A recent study argues that separators change rhythm-related properties that SDR does not capture.	Supports using a placement-aware metric.

The numbers come from the cited papers and benchmark pages and use different test setups, so they are not directly comparable with each other. Stage re-measures every model on its own held-out set (Chapter 26).

22.3 Triage-and-refine separation

Most of a song is easy for a modern separator. Errors concentrate in a minority of passages: dense choruses, loud drum fills under a vocal, reverb tails. Running a heavy ensemble on everything wastes compute on the easy part. Triage-and-refine runs a fast model everywhere, estimates where it probably failed, and spends heavy compute only there.



average cost per track = c_fast + q * c_heavy

q is the share of chunks the proxy flags. If q = 0.25, c_fast = 3 and c_heavy = 25 GPU-seconds, the average is 9.25 versus 28 for refining everything.

Figure 10. Triage-and-refine. The proxy decides which chunks get the heavy model.

The quality proxy

Ground truth is not available at run time, so the proxy has to be computed from the audio and the fast separation alone. Three cheap signals are combined by a small regression model trained on songs where the truth is known:

Signal	Definition	Cost
Consistency gap	c1 = || x - sum of stems ||^2 / || x ||^2 per chunk. Large values mean the model left a lot unexplained.	Almost free.
Cross-stem envelope correlation	For stem pairs that should be independent in a band b (for example vocals and drums), rho = correlation of their envelopes. High values suggest bleed. Real music shares rhythm, so this signal is calibrated, not trusted blindly.	Very cheap.
Pan and coherence structure	A clean stem has a consistent panning pattern; a stem with a large share of low-coherence, scattered-pan content is suspect (uses the analysis from Chapter 21).	Cheap; already computed.
Shift disagreement (optional)	Run the fast model on a copy shifted by a fraction of a segment, shift back, and measure the difference from the first result. Disagreement between two views of the same audio is a standard uncertainty signal.	One extra fast pass, so double the fast cost.

```
flag(chunk) = 1 if predicted_error(chunk) is in the top q of the song (or above a
    calibrated threshold)
predicted_error = f( c1, rho_ij, coherence statistics, optional shift disagreement )
    (a small gradient-boosted or linear model)
average cost per track = c_fast + q * c_heavy
Example: c_fast = 3, c_heavy = 25 GPU-seconds, q = 0.25 -> 3 + 0.25 * 25 = 9.25
    versus 28 for refining everything (about 3 times less)
q = 0.15 -> 6.75, about 4 times less
```

How to know the proxy works

- **Calibration.** On held-out songs with true stems, compute the true error per chunk and the proxy. The rank correlation between them is the first check; the target is at least 0.5 (hypothesis H10).
- **Capture rate.** Compare refining the proxy's top q chunks with refining the true worst q chunks (the oracle). Capture rate is the share of the oracle's gain that the proxy achieves.
- **Safety.** If the proxy is poor, the system flags more chunks, which costs more but is never worse in quality. The worst case is the ensemble-everywhere cost.
- **Seams.** A refined chunk is cross-faded into its neighbours over the overlap region so that switching models does not create audible steps.

22.4 Render-aware fine-tuning

This is the main machine-learning contribution. Separation models are trained to make each stem close to its true stem. But what listeners hear is the **rendered** mix, in which the stems are placed at different directions. A given stem error is much more audible when it lands at a place where the true content is absent than when it sits under a masking sound. Training on the rendered output teaches the model to avoid the errors that matter after placement.

```
Let R_pi be a linear renderer for a placement plan pi: R_pi(s) = sum over i of (
    h_{pi,i} * s_i ) (+ shared reflections)
Loss = lambda_s * sum_i || s_hat_i - s_i ||_1 (usual stem loss)
     + lambda_r * E over pi of MRSTFT( R_pi(s_hat), R_pi(s) ) (error on the
         rendered stereo output)
     + lambda_m * || x - sum_i s_hat_i ||_1 (consistency with
         the mix)
pi is sampled from the style templates with random jitter, so the model is accurate
    for any plausible placement.
MRSTFT = multi-resolution short-time Fourier transform magnitude loss
```

Because the renderer is just filtering, it is differentiable and cheap, and gradients flow through it to the separator. To keep training simple, the training renderer uses fixed placements per example, not the full pan-analysis masks (which depend on the output and are harder to differentiate).

Why this should help (hypothesis H7)

- **Audibility weighting.** The rendered loss is large for a vocal ghost leaking into a stem that is placed far to the side, and small for the same energy hidden under a loud centred sound.
- **Cross-stem interaction.** Stem losses treat each stem alone. The rendered loss sees what the errors of different stems do together.
- **Cheap to train.** Fine-tuning only the last layers on a small dataset is feasible on free GPU tiers split into resumable runs. The compute is an estimate of tens of GPU-hours in total, to be measured.
- **Clear test.** Compare a fine-tuned model with a baseline of equal SDR. If the rendered error and the blind-test preference improve while SDR stays level, the idea is confirmed.

22.5 Distillation: moving the ensemble's quality into one fast model

1. **Teacher.** The best available pipeline: heavy models, shifts and the render-aware fine-tuned model, run once offline.
2. **Student.** A fast model of the SCNet class, a few million to a few tens of millions of parameters.
3. **Data.** Ground-truth multitracks where available, plus teacher outputs on audio you may use (your own productions and openly licensed music, never Spotify content or streams).
4. **Loss.** The student matches the teacher's stems and, where known, the true stems, plus the rendered loss of 22.4.
5. **Gate.** The student must lose no more than about 0.2 dB against the teacher on the rendered metric and cut compute by at least a factor of three.

> **Limits of distillation**
>
> Distillation is capped by the teacher: the student cannot be better than what it copies. It also inherits legal questions from the teacher's weights. For research and the demo that is acceptable. For a product, the teacher must itself be built on clearly licensed weights and data.

22.6 Quantisation and export

Step	Method	Reason	Gate
GPU	Half precision (FP16)	Large speed-up, small risk.	No audible change; change in SDR and rendered score under 0.05 dB.
CPU	INT8 for the middle layers, with the input transform and the output layers kept in FP16 or FP32	The study above shows input and output activations are the sensitive parts.	Loss of at most 0.1 dB on both metrics.
Training for it	Quantisation-aware fine-tuning with distillation if post-training quantisation loses too much	Recovers most of the lost accuracy.	Same gate.
Calibration	A few hundred representative chunks across genres	Quantisation ranges depend on the data.	Calibration set kept apart from the test set.
Format	ONNX with ONNX Runtime	Portable across servers, laptops and phones.	Outputs match PyTorch within tolerance.

22.7 Chunking: the arithmetic that decides speed

Long songs are processed in overlapping segments. Hybrid Transformer Demucs works on 7.8-second segments. The overlap lets neighbouring segments be blended without seams, but every overlapped second is computed twice:

```
cost multiplier = 1 / (1 - overlap)
overlap 25 percent -> 1.33 times the work; overlap 10 percent -> 1.11 times;
    overlap 50 percent -> 2 times
Shift averaging with N shifts multiplies the cost by N.
Plan: fast tier uses about 10 percent overlap and no shifts; refined chunks use 25
    percent overlap and a few shifts.
```

22.8 The metric that matters: render-weighted leakage

```
RLS_i = 10 * log10( || R_pi(s_hat_i) - R_pi(s_i) ||^2 / || R_pi(s_i) ||^2 )
RLS = average over stems i and over five placements pi. Lower (more negative) is
    better.
Reported next to SDR, SIR and SAR, never instead of them.
```

RLS answers the question that matters for this product: how wrong does the rendered result sound, not how wrong is the isolated stem. Every model comparison in the project reports both kinds of number, so no result depends on a metric that was invented to flatter it.

22.9 The ladder and its gates

Level	What is added	Gate to keep it
S0	HT-Demucs baseline, standard settings	The reference
S1	A fast SCNet-class model	Within 0.3 dB of S0 on SDR and RLS at no more than half the compute
S2	Triage-and-refine on top of S1	At least 90 percent of the gain of refining everything, at no more than 40 percent of its compute
S3	Render-aware fine-tuning	At least 0.5 dB lower RLS than S2 at equal compute, and preferred in blind tests
S4	Distillation and quantisation	Within 0.2 dB of the teacher, at least three times cheaper, and a clear licence story

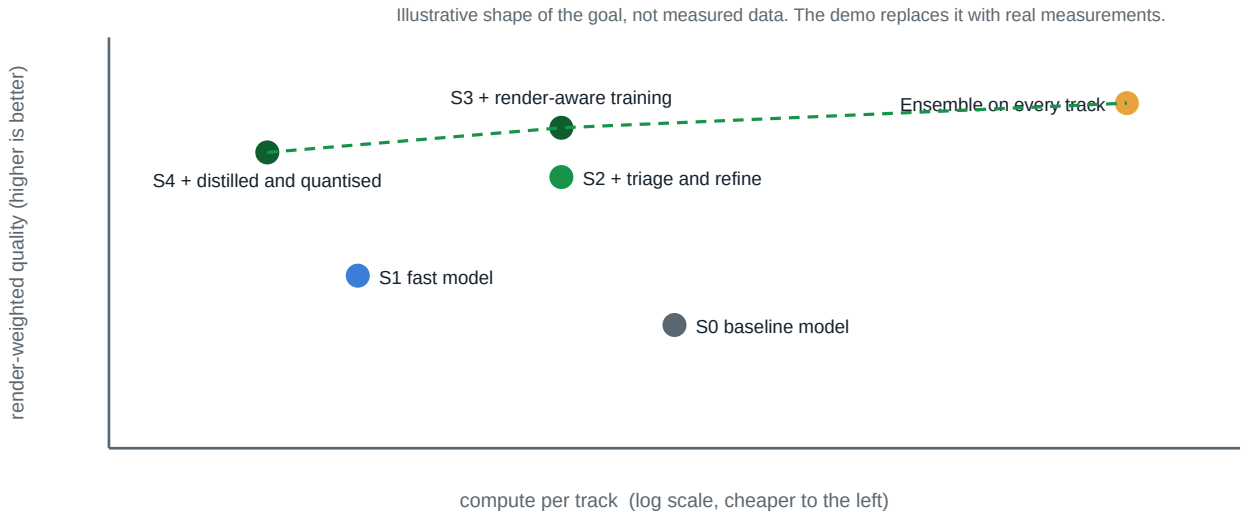


Figure 11. The target shape: S2, S3 and S4 sit above and to the left of simply running an ensemble on everything. The positions are illustrative until measured.

The thresholds in the gate column are targets set in advance, not predictions. If a level misses its gate it is removed or reworked, and the project continues on the previous level, which is always a working system.

22.10 Data and licences for training

Source	Use	Licence note
MUSDB18-HQ (150 songs; 100 for training and 50 for testing in the usual split)	Fine-tuning and the main test set	Released for research; check terms before any product use.
MoisesDB and similar multitrack sets	More training variety	Check each licence.
Your own multitrack sessions and willing collaborators	Training, testing and public demos	Safest; get written permission from collaborators.
Openly licensed audio (per-track Creative Commons terms)	Teacher outputs for distillation, and listening tests	Licences differ by track; keep a manifest.
Spotify audio, previews or API data	Never	Spotify's developer terms bar using Spotify content to train models, and exceptions are not made for research.

Decision: a fast SCNet-class model with triage-and-refine, then render-aware fine-tuning, distillation and quantisation	
Choice	Use the fast model everywhere, spend heavy compute only on flagged chunks, train on the rendered result, then compress.
Why	Each step attacks compute or audible error directly, each has a gate, and the combination moves the efficiency frontier instead of chasing a single leaderboard number. The fast model keeps the whole system affordable and portable.
Drawbacks	A proxy can mis-flag chunks (costing compute, not quality). Render-aware training adds engineering. The gain from fine-tuning on a small dataset is uncertain, which is why it is a hypothesis with a gate.
Alternatives	Run the best ensemble on everything (best quality per track, highest cost), use only the lightest real-time model (lowest cost, lower quality), or build a new architecture (high risk, high effort, not needed to test the ideas).

23 Languages and stack for a light, portable engine

Lightweight is a design constraint, not a polish step. The rule is: **the part that runs on the listener's device must be small, fast and portable, and everything heavy runs offline.** This chapter fixes the languages and sets budgets that the build has to meet.

23.1 Where each language goes

Layer	Language and tools	Why	Alternative considered
Real-time core (PASR, Lite)	Rust, as a library called through a plain C interface	One code base for the browser (WebAssembly), Android and iOS; no garbage collector, so no audio glitches; memory safety.	C++ (equally fast, wide audio ecosystem, but memory-unsafe and a heavier web toolchain).
Reference implementation	Python with NumPy and SciPy	Easy to read and to test; it is the oracle the Rust core must match.	Writing only the Rust version (faster, but no independent check).
Offline engine and orchestration	Python 3.12	All audio and machine-learning tooling lives here.	None needed.
Separation inference	ONNX Runtime (CPU, and GPU when available)	One portable model format, quantisation support, small runtime, no training framework at run time.	PyTorch at run time (heavier); TensorRT (fast, NVIDIA only).
Training	PyTorch on free GPU notebooks	The model code is in PyTorch.	JAX (less ecosystem for these models).
Python-to-Rust bridge	PyO3 with maturin	Lets Python tests call the Rust core directly for equivalence checks.	A command-line bridge (slower, clumsier).
Web demo	TypeScript, Svelte, Vite; Rust compiled to WebAssembly inside an audio worklet	Real-time audio off the main thread; tiny bundle; works on any phone browser.	Pure JavaScript DSP (slower and harder to keep identical to the core).
Android client (optional)	Kotlin with Media3 for playback; the Rust core behind a native audio callback for the real-time path	Shows the real-device path.	Flutter or React Native (heavier, less audio control).
HRTF data	A Python tool that converts open SOFA files into one small binary pack	SOFA is a scientific container that is heavy to read at run time; a pre-baked pack removes that dependency.	Reading SOFA files directly on the device.

23.2 Budgets the build must meet

Budget	Target	How it is measured
WebAssembly core size	At most 300 KB compressed	Build report in CI.
HRTF pack size	At most 300 KB	File size.
First load of the demo page (code only)	At most 500 KB compressed	Network panel on a throttled connection.
Lite CPU load	At most 5 percent of one mid-range phone core	Audio callback time divided by buffer duration, on a real phone.
Memory for Lite	At most 20 MB	Browser and Android profilers.
Start-up of spatial playback	At most 300 ms after the first audio	Timer in the demo.
Latency	About one window (46 ms) plus the device's own buffer	Loopback measurement or calculation.
Quantised fast separation model	A target under 50 MB; offline only	File size.
Energy	Measured against plain playback	Battery statistics over a long listening session.
Cost per track (offline)	Measured and published	Per-job timers and public GPU prices.

> **Targets, not promises**
> These targets are set in advance as engineering goals, not claims. The estimate in Chapter 21 (about 55 million operations per second) suggests the CPU budget is comfortable, but the real figure depends on the phone, the browser and the implementation, and has to be measured.

23.3 The one-file HRTF pack

- A Python tool reads open SOFA files, picks the chosen subject, and writes a single compact binary: for each of the 13 sectors across a range of azimuths, a minimum-phase impulse response pair and an interaural delay, plus a small set of directions for reflections.
- Minimum-phase filters of a hundred or so taps, with the delay stored separately, keep the data small and the filtering stable.
- A few subject profiles can be shipped and chosen by the listener; each pack is a few hundred kilobytes at most.
- The licence and attribution of each source dataset travel inside the pack's header, so credit is never lost.

23.4 Keeping Rust and Python identical

1. The Python reference renders a fixed test clip with a fixed plan and stores the result as a golden file.
2. The Rust core must match the golden file within a tiny tolerance (about one part in 100,000 in 32-bit floating point).
3. Properties such as length, silence handling, energy split and alignment are tested on both.

4. Any change to the algorithm changes the reference first, then the golden files, then the Rust core. Nothing is changed in only one place.

23.5 Repository layout in v2

```
stage/
  reference/ Python reference renderer and analysis (the oracle)
  stage-core/ Rust crate: STFT, pan analysis, sectors, HRTF filtering,
      reflections, tail, timbre, limiter
  tools/hrtf-pack/ Python: SOFA to compact binary
  engine/ offline engine from the first specification (analysis,
      separation, plan, guard)
  models/ ONNX files and a manifest with licences and checksums
  web/ TypeScript demo and the audio worklet
  android/ optional client
  configs/ schemas/ tests/ docs/ scripts/ as before
```

Decision: Rust for the real-time core, with a Python reference as the oracle	
Choice	The part that must run on the listener's device is written in Rust and compiled to WebAssembly for the browser; a NumPy version is the reference.
Why	Portability and safety. One source runs in the browser and on phones, there is no garbage collector to cause glitches, and the independent Python version catches mistakes. It also reuses skills from a Rust and WebAssembly acoustics project.
Drawbacks	Rust has a learning curve, and audio-specific libraries are fewer than in C++. Mitigation: the core is small (STFT, per-bin maths, short filters), so it needs few libraries, and the Python core remains a working fallback for the demo.
Alternatives	C++ is a fully valid choice with the same architecture, and may suit you better if you want C++ practice; it costs more in safety and in web tooling. Python only would be simplest but cannot run in real time on a phone.

24 The demo v2: built to impress in five minutes

A busy engineer decides in minutes. The demo is designed so that the strongest evidence comes first, everything can be tried without an account, and the claims on the page can be checked.

24.1 The five-minute path

Minute	What the visitor does	What they learn
0 to 1	Drops any song from their own device onto the page. It plays spatially within a second, running in their browser.	It works on any song, with no upload, no server and no special phone.
1 to 2	Flips between Stereo and Spatial while watching the X-ray view.	The change is real, aligned and level-matched; they can see where each part of the mix is placed.
2 to 3	Plays the showcase tracks in Standard and Studio tiers and switches modes.	The richer tiers sound better and are still faithful to the original.
3 to 4	Opens the metrics panel: fidelity checks, timbre deviation, CPU load, and the cost per track.	The engine is measured, light and cheap.
4 to 5	Takes the blind test or reads the results and the limitations.	The claims are tested, and the weaknesses are stated.

24.2 What is on the page

Element	Content	Why it earns its place
Drop zone	Any local audio file; processing happens in the browser with the Lite tier. Nothing is uploaded.	Proof of 'any song', zero server cost and no copyright or privacy issue.
Player	Stereo and Spatial switch, mode chips, depth and width sliders, level-matched.	The core experience.
X-ray view	A top-down animated view driven by the real sector energies from the analysis: where each part sits and where the ambience goes.	Makes the method visible and shows it is not a black box.
Showcase tracks	Six to eight tracks, pre-rendered in Standard and Studio, rights-cleared.	The best audio quality; the reliable part of the demo.
Metrics panel	Live CPU load of the audio worklet, timbre deviation, fidelity checks, size of the code that was downloaded, and measured cost per track.	Evidence for lightness and efficiency, in the viewer's own browser.
Efficiency chart	Quality against compute for each separation level, from measured data.	Shows the compute story honestly.
Blind test	Eight clips, randomised, anonymous.	Evidence for the sound.
Limitations	A plain list, including where Lite is weaker than Studio.	Credibility.
Reproduce	One command and a manifest that rebuild the showcase from licensed sources.	Lets others check.

24.3 Designing the demo not to fail

- **Graceful fallbacks.** If WebAssembly or the audio worklet is unavailable, the page falls back to the pre-rendered showcase files, and says so.
- **Static hosting.** Everything is static files; there is no server to run or to crash. Check the free host's published size and traffic limits before choosing where large audio files go.
- **Data-light.** Code is small, audio loads only on request, and a data-saver option uses compressed versions of both sides of the A/B test.
- **Cross-device checks.** Test on at least two phones and two browsers before sharing the link.
- **Honest labelling.** The page states that it is independent and not affiliated with any streaming service, and that automated spatial mixes are not made by a human mix engineer.

24.4 The video and the write-up

- The two-minute video follows the five-minute path, opening with a song dropped into the page.
- The write-up has three parts: a one-page summary, the method (Chapters 21 and 22), and the evidence with limitations (Chapter 26).
- Both link to the live demo and the repository.

25 Reaching Spotify: a realistic strategy

You cannot control whether a large company replies. You can control how easy the work is to find, trust and evaluate. This chapter lists what was verified about Spotify's rules and builds a plan that does not depend on a single response.

25.1 Facts that shape the strategy

Fact	Source type	What it means for you
Spotify's terms say feedback and ideas submitted to it are not confidential and may be used without payment.	Spotify's terms, as summarised by third-party policy trackers (read the original text yourself).	Do not rely on secrecy or on sending a private pitch to protect an idea. Your protection is public, dated, real work and your own reputation.
Spotify's developer terms bar using Spotify content, including API data, to train machine-learning models, and Spotify says it does not make individual exceptions.	A reply from Spotify staff in its developer community.	Never use Spotify audio, previews or API data in training, testing or the demo.
The Spotify Community Idea Exchange contains long-running requests for spatial audio, with many supporting comments.	Spotify Community threads, mainly from 2021.	Evidence of listener demand (anecdotal). Cite it as such, not as a statistic.
Spotify engineers have open-sourced audio tools: Pedalboard (GPL v3) and Klio (Apache licence).	Public repositories.	A natural bridge: contribute to them, and show you understand how their audio teams work.
Large audio companies hold patents in binaural virtualisation, including structures that use feedback delay networks.	Public patent records.	Check freedom to operate before any product claim, and avoid copying patented structures.

25.2 The plan

1. **Build the evidence first.** Reach out only after slice 5, with a working demo, a result with an honest interval and the measured cost per track. An early message with only an idea is the weakest form of the pitch.
2. **Publish openly.** A repository with a permissive licence for the DSP core makes it easy for anyone to evaluate and signals confidence. A licence does not protect the idea from being used, but publication gives you a dated record of authorship.
3. **Write for engineers.** A short technical report with the method, the maths, the measurements and the limitations. Put it on a preprint server, which is free.
4. **Make contact the right way.** Short, specific messages to engineers and researchers in audio, personalisation and machine learning, with one link to the demo and one sentence on what is measured. Do not attach files.
5. **Contribute.** A real contribution to Pedalboard or Klio (a bug fix, documentation, a benchmark) builds credibility in the exact community you want to reach. Remember Pedalboard is GPL v3, so contributions are under that licence.
6. **Use the community channels.** A polite, relevant comment with the demo link in the existing spatial audio idea threads, following the community guidelines.

7. **Share where audio engineers gather.** Student and professional audio communities, your IEEE chapter, and music technology groups. Their feedback improves the work whatever Spotify does.
8. **Aim at roles, not only at the feature.** The project is the strongest possible application evidence for audio machine-learning roles at Spotify and at other companies.

25.3 What a reviewer will look for

Question a reviewer asks	Where the answer lives
Does it work on my phone, right now?	The drop-any-song page.
Is the claim measured?	Metrics panel, blind-test results, intervals.
Is it light enough to ship?	Size and CPU budgets, measured.
What does it cost at scale?	Cost per track and the storage analysis (Chapters 12 and 13).
How would it fit our pipeline?	Integration surface and platform diagram (Chapter 3).
What about artists and rights?	The consent model and approval flow.
Are the weaknesses understood?	The limitations lists.
Can I reproduce it?	One command, manifests, licences.

25.4 Possible outcomes and what to do

Outcome	Likelihood (judgement)	What to do
No reply	Most likely	Keep improving and sharing; the portfolio value is unchanged.
A polite no or a form answer	Likely	Ask what would make the work more useful; thank them.
A short technical conversation	Possible	Prepare the Chapter 15 answers; offer a measured comparison on their questions.
Interest in the idea or in you	Less likely	Be ready to discuss rights, cost and integration; keep claims within the evidence.

> **The outcome you control**
>
> The goal that does not depend on anyone replying: a public, reproducible, honestly measured spatial audio engine that runs in a browser on any song. That is valuable to you, to other companies you target, and to the research community, whatever Spotify does.

26 Experiments, gates and the v2 roadmap

This chapter turns every claim in Part II into an experiment with a pass condition decided in advance. It is the mechanism that keeps the plan honest, and the reason the earlier weak spots stop being open risks.

26.1 Hypothesis register

ID	Claim	Test	Gate	If it fails
H1	PASR-Lite is preferred over a generic virtualiser (B1).	Blind A/B, 10 then 30 listeners, level matched.	Lower end of the 95 percent interval above 50 percent.	Revisit widening factor and room; reduce intensity.
H2	The timbre bound reduces coloration without losing direction.	ABX and preference with and without compensation; TD metric.	TD at most 1.5 dB; compensation preferred or equal.	Loosen the bound or shrink the limit.
H3	Role-based distance beats one room for everything.	Blind A/B on vocal-led and drum-led tracks.	Role-based preferred.	Use a simpler two-bus design.
H4	Masking-aware placement improves clarity or preference.	Blind A/B with and without; vocal-intelligibility task.	Preferred or equal at no extra artifacts.	Remove it.
H5	Order-preserving widening beats template placement.	Blind A/B against first-document templates.	Preferred or equal.	Keep templates as a mode.
H6	Triage-and-refine keeps most of the ensemble's gain at low cost.	Capture rate and cost on held-out songs.	At least 90 percent of the gain at no more than 40 percent of the cost.	Raise q; or use the fast model only.
H7	Render-aware fine-tuning lowers rendered error at equal SDR.	RLS and blind test against a baseline.	At least 0.5 dB lower RLS, preferred.	Drop it; keep S2.
H8	Lite meets the size and CPU budgets.	Measure on real devices.	All budgets in 23.2 met.	Optimise or reduce sectors.
H9	Lossy encoding keeps the spatial effect.	ABX lossless against high-bitrate lossy.	Not reliably distinguishable.	Report the limit; recommend lossless.
H10	The quality proxy tracks true quality.	Rank correlation on held-out songs.	At least 0.5.	Flag more chunks; improve features.

26.2 Added risks and their gates

Risk	Early warning	Response
Pan-anchored widening sounds smeared on some mixes	Listeners report vague images	Coherence gate stricter; reduce widening for low-coherence material.
Time-panned or spaced-microphone recordings break the level model	Low coherence across most of the song	Treat as ambient-dominated; gentler profile.
Old mono recordings have no image to preserve	Zero side energy	Ambient-only enhancement with a reduced effect.
Real-time core misses its budget on some phones	High callback load	Fewer sectors, shorter filters, a lower-cost tail.
Proxy flags the wrong chunks	Low capture rate	More flags, better features, or fall back to refine-all for showcase tracks.
Patent exposure in virtualiser structures	A match found in a review	Change the structure; keep the demo non-commercial; seek advice before any product claim.
Blind-test sample too small	Wide intervals	Report them honestly; recruit more; narrow the questions.

26.3 The first ten working sessions, reordered

1. Create the repository and layout; install tools once on a good connection; write the first decision records.
2. Write the Python reference for the stereo analysis: STFT, panning factor, coherence, primary and ambient split. Test that primary plus ambient equals the input.
3. Add the thirteen sectors and the widen warp; test that order is preserved and energy is conserved.
4. Convert one open HRTF subject into the compact pack; filter test tones at known azimuths and measure the delays and levels.
5. Add the three reflection buses and a simple late tail; listen to a short clip and write down what is wrong, specifically.
6. Add the timbre compensation and the guard checks; compute timbre deviation on three clips.
7. Render five clips in different styles; compare each with the original and with a generic virtualiser (B1).
8. Build the A/B page for those clips; run the first informal blind test with three or four people.
9. Fix the largest problem found; re-render; record before and after.
10. Write the three-line summary of slice 1 and decide whether to continue to slice 2.

26.4 The v2 definition of done

- Lite runs in a browser on a real phone, in real time, on any dropped song, within the budgets.
- At least six showcase tracks in Standard and Studio tiers, rights-cleared, passing the guard.
- Blind-test results with intervals for H1 to H5, and the separation experiments for H6, H7 and H10, reported whether or not they succeeded.
- The efficiency chart and the cost per track from measured data.

- A public repository with a permissive licence for the core, a manifest of every audio file and every model with its licence, and a one-command reproduction.
- The write-up and the video.

> **Why failure is safe**
> If a gate fails, the project does not stop. It continues on the previous level, which is always a working, demonstrable system, and the failure itself becomes a documented result. That is why this plan can be ambitious without being fragile.

A Appendix A: glossary

Term	Meaning
Apache Beam	A framework for batch and streaming data pipelines that runs on several back ends. Klio is built on it.
Audio fingerprint	A hash or signature of decoded audio, used to recognise the same recording in different places.
Bag of models	Several models used together; Demucs's fine-tuned version uses one specialist per stem.
Cascade	A design where a cheap method runs first and a costlier one runs only if needed.
Distillation	Training a small model to imitate a larger one.
Feedback delay network (FDN)	A reverb design made of delay lines fed back into each other.
Fine-tuning	Continuing to train an existing model on new data.
Golden file	A stored expected output that new results are compared with.
Idempotent	Safe to run more than once: the same inputs always give the same result.
ONNX	An open model format that lets a model run in many environments without its original framework.
Quantisation	Storing and computing a model with lower-precision numbers to make it smaller and faster.
Residual	What remains of the mix after the separated stems are subtracted.
Spot instance	Cloud computers offered at a discount that can be taken back with short notice.
Tiering	Moving data between storage classes that trade price against access speed.
TensorRT	NVIDIA software that optimises neural networks for fast inference on its GPUs.
Wilson interval	A method for confidence intervals on proportions that behaves well for small samples.

B Appendix B: one-page checklists

Before you start building

- Repository and folder layout created.
- Decision records started (copy the ones from this document).
- Models, datasets and packages downloaded once and cached.
- A manifest of test music with the source and license of every file.
- A baseline benchmark on your laptop written down.

Before you share anything publicly

- Every audio file is yours, openly licensed or used with permission.
- The disclaimer about independence is on the page and in the video.
- Attributions for HRTF datasets and libraries are included.
- The listening test collects no personal data and says so.
- Limitations are written down in plain words.

Before you answer a hard question

- Is the claim measured, estimated or assumed? Say which.
- Can you point to the chapter or the measurement behind it?
- If the honest answer is 'I do not know yet', say what you will do to find out.

C Appendix C: sources

The following sources were consulted in November 2026. Prices, quotas and licenses change, so recheck anything that matters before relying on it.

Spotify and market

- Spotify Q3 2026 results (Form 6-K exhibit on sec.gov, and the company's earnings announcement); coverage on hypebot.com and newindustryfocus.com.
- Spotify for Artists help, 'Audio file formats for Spotify': support.spotify.com/artists/article/audio-file-formats/
- Spotify lossless launch coverage (September 2025): soundguys.com, audioxpress.com.
- Music Business Worldwide on MIDiA subscriber shares; sqmagazine.co.uk market statistics.
- Luminate 2024 findings as summarised by RouteNote; Dolby-commissioned US survey, December 2021; Futuresource Consulting, 2021.
- Spotify engineering blog, 'It's All Just Wiggly Air': engineering.atspotify.com; Klio on github.com/spotify/klio; Pedalboard documentation at spotify.github.io/pedalboard.

Separation models and performance

- Demucs repository notes and community forks (github.com/facebookresearch/demucs, adefossez/demucs, openmirlab/demucs-infer); discussion on the license of the pretrained weights on huggingface.co/adefossez/HTDemucs.
- MVSEP algorithm benchmarks: mvsep.com/algorithms; Mel-Band RoFormer paper on arxiv.org/abs/2310.01809; training framework by ZFTurbo on github.com.
- ONNX and TensorRT exports of Demucs and their benchmarks on Hugging Face and PyPI (demucs-onnx, Demucs_v4_TRT, demucs-mlx).

Platform, data and prices

- Android Open Source Project, 'Spatial audio and head tracking': source.android.com/docs/core/audio/spatial
- HRTF data: ARI database (oeaw.ac.at), SADIE II (University of York), Viking HRTF dataset (Zenodo), and a published comparison of public HRTF datasets on arxiv.org/abs/2210.15196.
- Google Cloud Storage list prices as summarised by several 2026 pricing guides; GPU pricing from getdeploying.com and provider pages; free GPU notebook limits from several 2026 comparisons.

> **How to read the sources**
>
> Where this document says 'verified', the statement matched at least one source above at the time of writing. Where it says 'estimate', the calculation is shown. Where it says 'open question', the document says how to settle it.

D Appendix D: sources added in v2

Sound design and stereo analysis

- Avendano and Jot, frequency-domain stereo upmix and ambience extraction; Barry and colleagues on azimuth discrimination and resynthesis; overview of lead and accompaniment separation (arxiv.org/abs/1804.08300).
- Kraft and Zolzer, stereo signal separation and upmixing by mid-side decomposition in the frequency domain (DAFx-15).
- He, Tan and Gan, primary-ambient extraction for spatial audio; efficient and PCA-based primary-ambient decomposition papers; style-transfer upmixing (arxiv.org/abs/2203.12053).
- Headphone externalisation and reflections: Aalto thesis on headphone sound externalisation; studies on early-reflection cues and on binaural music rendering with varying realism; overview of natural sound rendering for headphones.
- Patent records describing feedback-delay-network binaural virtualisers, for freedom-to-operate awareness only.
- Atmos mixing practice and the Off, Near, Mid, Far binaural render modes: Avid, Production Expert, Abbey Road Institute and Dolby delivery guidance; a mixer's post on Apple's developer forums about how Apple's re-render treats those settings (one user's observation).

Separation and efficiency

- SCNet: Sparse Compression Network for Music Source Separation (arxiv.org/abs/2401.13276) and its official code repository.
- Mel-Band RoFormer for music source separation (arxiv.org/abs/2310.01809); MVSEP benchmark tables.
- Towards practical real-time low-latency music source separation (arxiv.org/abs/2511.13146); fully quantised neural networks for audio source separation (IEEE Open Journal of Signal Processing, 2024); Music Source Separation in the Waveform Domain (arxiv.org/abs/1911.13254).
- Beyond SDR: how music source separation reshapes rhythm-relevant signal properties (arXiv, 2026).

Spotify's rules and community

- Spotify Terms and Conditions (read the current text directly); third-party summaries of its feedback and ideas clauses.
- Spotify for Developers community reply on using Spotify content to train models.
- Spotify Community Idea Exchange threads requesting spatial audio.

> **What is established and what is proposed**
>
> The formulas in Chapters 21 and 22 follow standard results in the cited literature (least-squares primary estimate, tangent law, critical distance, overlap cost). The specific combination, constants and thresholds are design choices to be tested, not published results.